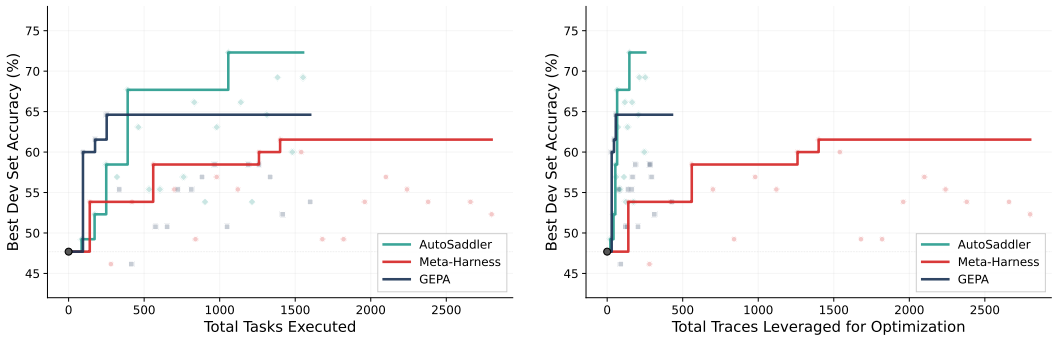
AutoSaddler：基于智能体执行轨迹持久更新的自动化支架优化

Sungho Park[1]* **Wonjoong Kim**[2]* **Rongyuan Tan**[3]* **Jue Zhang**[4]† **Wook-Shin Han**[1]†
Pengfei Gao[4] **Chanyoung Park**[2] **Yongqiang Yao**[4] **Rao Fu**[4] **Elsie Nallipogu**[4]
Qingwei Lin[4] **Saravan Rajmohan**[4] **Dongmei Zhang**[4]

[1]POSTECH [2]KAIST [3]南方科技大学 [4]微软
juezhang@microsoft.com wshan@dblab.postech.ac.kr

摘要

LLM 智能体在长时程任务上仍不可靠：在长时间交互过程中，微小的局部失败可能不断累积，最终导致整个任务失败。尽管外部支架可以显著提升鲁棒性，但支架设计仍是一个依赖人工且成本高昂的过程，需要在由提示词、工具配置和控制逻辑构成的巨大空间中进行搜索。我们提出 **AutoSaddler**，这是一种自动支架优化框架，它将支架改进表述为离线学习问题，并利用来自小批次的失败信号迭代更新支架。AutoSaddler 将失败轨迹诊断、把支架视为代码的结构化补丁生成，以及基于验证的更新选择结合起来。在 GAIA2、SWE-Bench Pro 和 Terminal-Bench 2.0 上的实验表明，与相应的基础支架相比，AutoSaddler 显著提升了智能体性能，分别取得了 **9.0**、**9.6** 和 **10.0** 个百分点的增益。消融研究进一步表明，有效的支架优化得益于三个要素：采用深度调试而非浅层反思，采用有针对性的修改而非不受约束的编辑，以及采用关注泛化的选择而非针对特定轨迹的修复。综合来看，这些结果表明，自动支架优化是构建性能更强、更可靠的智能体系统的一条很有前景的路径。项目网站与代码将发布于 此链接。



(a) 计算效率 (b) 学习效率

图 1: **GAIA2 上优化性能与效率的比较。(a)** AutoSaddler 仅用总计 ∼**1,000** 次任务执行便达到 **72.3%** 的开发集准确率；相比之下，GEPA 和 Meta-Harness 尽管消耗了 ∼**2,800** 次任务执行，性能却分别在 **64.6%** 和 **61.5%** 达到饱和。**(b)** 若以用于优化的执行回合数衡量，AutoSaddler 仅消耗 147 条轨迹便取得最佳性能，比 Meta-Harness（1,400 条轨迹）少约 ∼**10**×。

*本工作于微软实习期间完成。
†通讯作者。

预印本。

1 引言

近年来，大语言模型（LLM）的能力迅速提升。尽管取得了这些进展，它们仍表现出“锯齿状智能”：在某些任务上表现出色，却在另一些看似密切相关、甚至更简单的任务上表现不佳。这种不均衡性给智能体应用带来了根本性的可靠性挑战，在自主、多步和长时程场景中尤为如此，因为成功取决于智能体在许多连续决策中始终保持足够的能力。为应对这一问题，一类新兴工作尝试在 LLM 外部构建支架层，使智能体在长时程任务中的行为更加鲁棒 [61, 3, 5, 31, 21]。经验证据表明，设计良好的支架能够显著提升智能体性能 [21]。

然而，手工调优支架既耗时又难以扩展。它需要探索一个巨大的设计空间，其中包括提示词规范、工具配置以及其他系统级选择。此外，评估每个候选支架的成本也很高，因为智能体可能需要执行许多步骤，任务最终是成功还是失败才会变得明确。分析由此产生的长时程轨迹本身也并非易事，往往需要投入大量人工。这些挑战使人们迫切需要自动化支架优化方法，以便在更换底层 LLM 或将智能体部署到新领域时高效地调整支架。

在本工作中，我们将自动支架优化表述为一个离线学习问题，并提出 **AutoSaddler** 框架，利用成批训练任务产生的失败信号迭代改进支架。在每次迭代中，我们首先根据累积的探索历史构造候选支架，并在当前小批次的训练样例上对其进行评估。随后，我们诊断失败的智能体轨迹，并将支架本身视为代码，利用诊断所得见解指导补丁生成。对于每个被接纳的支架更新，我们还会在验证集上进一步评估其泛化能力。这些组成部分共同使 AutoSaddler 能够以有原则、可扩展的方式优化支架，同时推动所产生的更新超越已观察到的训练样本，实现更广泛的泛化。

我们在三个智能体基准上评估了 AutoSaddler：GAIA2 [10]、SWE-Bench Pro [9] 和 Terminal-Bench 2.0 [28]。在所有三个基准上，AutoSaddler 都显著提升了长时程智能体的性能。与相应的基础支架相比，它在 GAIA2 上提升 **9.0** 个百分点，在 SWE-Bench Pro 上提升 **9.6** 个百分点，在 Terminal-Bench 2.0 上提升 **10.0** 个百分点；相较最强的自动化基线，则分别提升 **7.4**、**4.4** 和 **6.7** 个百分点。消融实验和定性分析进一步支持了 AutoSaddler 的设计依据，并表明有效的支架优化依赖三个关键要素。第一，**深入诊断**是必要的，因为长时程失败需要深度调试，而不能仅对智能体执行失败进行浅层反思。第二，**结构化干预**十分重要，因为庞大且多样的支架空间更适合有针对性的修改，而非不受约束的编辑。第三，**关注泛化的选择**至关重要，因为要提升整个任务分布上的性能，就必须保留具有广泛用途的更新，而不是仅仅修复单条轨迹。

我们的主要贡献概括如下：

- 我们将自动支架优化表述为一个涵盖提示词、工具和中间件的离线学习问题，并利用智能体执行轨迹中的失败信号开展优化。
- 我们提出 AutoSaddler，将基于证据的诊断、结构化补丁生成和关注泛化的选择相结合，以产生持久有效的支架更新。
- 我们在三个具有挑战性的智能体基准上开展了广泛实验，证明了所提出方法在提升长时程智能体性能方面的有效性。

2 相关工作

自动提示词优化。 基于 LLM 的系统自动优化已在提示词优化的语境下得到广泛研究，其中包括无梯度搜索 [62]、文本梯度与小批次方法 [36]、感知历史的优化与进化优化 [47, 14, 1]、基于规划的搜索 [43]、编程抽象 [18]、自动微分式优化 [50]，以及贝叶斯多提示词调优 [32]。尽管 AutoSaddler 借鉴了这一研究脉络，但支架优化的范围比提示词优化更广：长时程轨迹往往需要比简单反思更深入的诊断，而且搜索空间横跨提示词、工具和运行时控制逻辑。

自进化智能体与基于经验的改进。 从失败轨迹中学习与基于经验的智能体改进 [60] 和自进化智能体 [2] 密切相关。在这些工作中，经验被用来构建程序库 [6]、创建工具 [37, 45]，以及构建记忆、技能或知识库 [33, 19, 29]。与这些通常采用在线持续学习的方法不同，我们关注的是旨在实现跨环境泛化的离线支架优化。

一些既有工作还进一步强调自指性，即元智能体是否与任务智能体相同，例如 Darwin-Gödel Machines [54, 55]、Gödel agents [49] 和 Huxley-Gödel Machines [42]。与之不同，我们不施加此类约束，因为我们的目标是在有限的执行回合预算下改进任务智能体的外部支架，而不是对自指动力学进行建模。

智能体系统与支架优化。 基于 LLM 的系统外部层自动优化建立在一系列广泛的既有工作之上，其中包括 LLM 推理超参数调优 [41, 11]、工作流优化 [56, 64, 44]，以及将系统本身视为代码的系统优化 [59, 16]。最近，这一研究方向已扩展至对完整智能体支架进行优化 [22, 25, 34, 40]。其中一项代表性工作是 Meta-Harness [22]，它同样使用编码智能体来构建端到端优化流水线。相比之下，我们更强调结构化补丁生成，以及显式考虑泛化能力的系统化训练流水线。

近期同期研究 [3] 进一步从以下方向拓展了自动支架优化：可观测性驱动的进化 [23]、由诊断和薄弱点引导的修复 [7, 52]、贝叶斯配置搜索 [38]、历史经验再校准 [13]、自监督回顾式优化 [35]、面向泛化的约束进化 [57]、经验驱动的测试时适应 [17]，以及支架与模型权重的联合优化 [15, 8]。总体而言，这些工作探索了支架优化中彼此互补的不同方面，而 AutoSaddler 则强调深入诊断、结构化干预和关注泛化的选择三者的共同作用。

智能体轨迹的失败诊断与修复。 具有复杂工具交互的长时程智能体轨迹推动了失败诊断方面的研究 [58, 51, 26, 12, 53]。这些方法主要面向单条轨迹的诊断，并且往往假设轨迹能够容纳于 LLM 的上下文窗口之内，这限制了它们对超长轨迹和小批次诊断的适用性。我们则基于 Claude Agent SDK（CA-SDK） [4]，利用其文件访问与上下文管理能力。近期工作还研究了用于修复失败执行的轨迹级干预 [27, 63]。与这类面向特定任务的“热修复”不同，我们的目标是生成持久的支架修复，从而改善未来的行为。

LLM 驱动的进化搜索。 我们的候选项选择方法以探索历史为依据，使用 LLM 从先前候选项中选择并重组补丁，这与近期由 LLM 增强的进化算法一致 [1, 30, 39, 46, 24]。我们通过基于 CA-SDK 的候选项选择将这一思想具体化，而专门设计的 LLM 驱动进化算法则留待未来研究。

3 预备知识

智能体任务与执行轨迹。 我们考虑需要多步推理、重复使用工具以及与环境交互的智能体任务。令 $x \in \mathcal{X}$ 表示从任务分布 $\mathcal{T}$ 中采样得到的任务描述，并假设 $\mathcal{X}$ 中的任务实例是无状态且相互独立的。

在给定支架参数 θ 的条件下，在任务 x 上运行任务智能体会产生一个随机执行过程。我们记为 $(\tau, \hat{y}) \sim P_\theta(\cdot \mid x)$, 其中，$\tau$ 是执行轨迹，$\hat{y}$ 是最终输出。等价地，支架 H_θ 定义了条件执行分布 $P_\theta(\tau, \hat{y} \mid x)$。这种随机性涵盖了 LLM 采样、工具使用决策和环境交互所带来的随机因素。

优化空间：智能体支架。 尽管“支架”一词通常泛指 LLM 之外一切有助于智能体有效运行的部分，但其确切范围在学术界与工程界仍处于演变之中 [61]。在本工作中，我们遵循 LangChain 的支架工程实践 [21]，聚焦于三类支架参数，而不考虑记忆或技能整理等其他组成部分，因为我们的设定假定任务在很大程度上是无状态且相互独立的。形式化地，我们将优化空间定义为：

$$\theta = (\theta_{\text{prompt}}, \theta_{\text{tool}}, \theta_{\text{middleware}}) \in \Theta, \tag{1}$$

其中，θ_{prompt} 指定指令和系统提示词，θ_{tool} 定义可用工具及其接口，$\theta_{\text{middleware}}$ 表示运行时控制逻辑，包括钩子和智能体循环行为。

目标：预算约束下的优化。 我们的总体目标是最大化目标任务分布上的期望任务性能。令 $\mu(\hat{y}, y^*)$ 表示任务级指标，例如准确率或通过/失败判定，它通过将最终输出 $\hat{y}$ 与标准答案 y^* 进行比较来完成评估。由于运行配备支架的 LLM 智能体可能具有随机性，我们定义

$$J(\theta) = \mathbb{E}_{(x,y^*)\sim\mathcal{T}}\mathbb{E}_{(\tau,\hat{y})\sim P_\theta(\cdot|x)}[\mu(\hat{y}, y^*)], \qquad \theta^* = \arg\max_{\theta\in\Theta} J(\theta),$$

其中，θ^* 表示优化后的支架。

在实践中，$J(\theta)$ 无法直接观测，且每次执行都会消耗一个执行回合。给定执行回合预算 K，AutoSaddler 会在该预算内探索得到的候选支架集合 $\mathcal{V}_K \subset \Theta$ 上进行搜索。对于有限数据集 D，我们使用如下经验估计：

$$\widehat{J}_D(\theta) = \frac{1}{|D|}\sum_{(x_i,y_i^*)\in D}\frac{1}{R_i}\sum_{r=1}^{R_i}\mu(\hat{y}_{i,r}, y_i^*), \qquad (\tau_{i,r}, \hat{y}_{i,r}) \sim P_\theta(\cdot \mid x_i),$$

[3] 这些工作出现在本研究准备的最后阶段或随后的同行评审期间。

其中，R_i 是任务 i 的重复执行次数。AutoSaddler 会在执行回合预算内评估过的候选项中，返回*开发集*经验得分最高的候选项：

$$\hat{\theta}_{\text{AS}} = \arg\max_{\theta \in \mathcal{V}_{K,\text{dev}}} \widehat{J}_{D_{\text{dev}}}(\theta).$$

最终报告的测试性能为 $\widehat{J}_{D_{\text{test}}}(\hat{\theta}_{\text{AS}})$。

4 所提方法：AutoSaddler

下面介绍 AutoSaddler，这是一个用于自动优化 LLM 智能体支架的迭代框架。我们首先概述其工作流程，随后详细说明各个关键组件。

概述。 我们将支架优化建模为一个离线学习问题。这一建模反映了实际应用场景：智能体支架通常在开发阶段完成调优，之后才部署到生产环境。由于训练可能涉及大量任务，且基于轨迹采样的评估成本高昂，我们遵循标准机器学习实践以及自动提示词优化方面的既有工作 [36, 1]，采用小批量训练范式。

具体而言，我们将任务集 $\mathcal{X}$ 划分为训练集、开发集和测试集，分别记为 D_{train}、D_{dev} 和 D_{test}。如图 2 所示，每次迭代 n 首先在小批量 $B_n \subset D_{\text{train}}$ 上评估由 θ_n 参数化的当前支架 H_n（❶）。随后，工作流程进入**诊断–补丁会话**，分析小批量中的失败执行轨迹并推导结构化补丁 $\Delta\theta_n$，从而得到更新后的支架 $H'_n = H_n + \Delta\theta_n$（❷）。接着在同一小批量上验证打过补丁的支架（❸）；若 $\widehat{J}_{B_n}(H'_n) > \widehat{J}_{B_n}(H_n)$，我们便认为该补丁带来了小批量改进。满足这一准则时，我们进一步在 D_{dev} 上评估 H'_n，以估计此次更新能否泛化到已观察小批量之外（❹）。

无论补丁是否被接受，**反思会话**都会比较打补丁前后的轨迹，记录已修复、发生退化、仍然失败和仍然通过的样例（❺），并将由此得到的经验与分数存入 **EvoDAG**。EvoDAG 是一个用于追踪支架更新演化历史的有向无环图（❻）。随后，**演化会话**利用 EvoDAG 提出下一个候选支架 H_{n+1}（❼）。当轨迹采样预算 K 耗尽后，AutoSaddler 会从所有在 D_{dev} 上评估过的候选项中，返回开发集经验得分最高的候选项。之后，仅在留出测试集上对选定的支架评估一次，不再利用测试反馈对其做进一步修改。

这一工作流程与标准小批量学习相呼应，但针对文本形式的支架优化做了调整。诊断–补丁–验证步骤（❷ 和❸）扮演了文本梯度下反向传播的角色：由于文本错误信号无法像数值梯度那样得到自动检查，因此每次更新都需要显式生成假设、干预支架并进行经验验证。EvoDAG 与演化会话则通过积累反思信号，发挥类似优化器的作用，从而执行感知历史的支架演化。我们在附录 A 中详细说明这种类比及其差异。

诊断–补丁会话。 在这一会话中，小批量中的执行轨迹（包括成功和失败的运行）会被传递给*诊断–补丁智能体*，用于故障诊断和补丁生成。我们不将诊断与补丁生成分离，使智能体在生成补丁时能够充分利用诊断期间收集到的上下文信息。我们的诊断流程遵循严谨且基于证据的原则。除每条失败轨迹外，我们还向智能体提供支架代码库 θ_n 以及结构化指引，使其能够逐步检索相关轨迹细节，从而缓解智能体长轨迹带来的长上下文挑战。智能体以这些证据为依据，识别疑似根因，考虑其他可能的假设，并提出补丁生成建议。

补丁生成同样采用结构化方式。具体而言，智能体会生成一个针对一个或多个支架组件的*结构化补丁* $\Delta\theta_n$。我们不会允许其不受约束地编辑整个支架代码库，而是仅公开实现支架功能逻辑的源文件，同时禁止访问与评估或基准数据相关的代码。我们进一步将补丁空间划分为三个类别：*提示词*、*工具*和*中间件*，分别对应智能体支架的不同层次。表 1 列出了各类别的子类型及其说明。

我们进一步将这些补丁类型归为两个更高层次的组别：**能力补丁**和**引导补丁**。能力补丁修改可执行代码或编排逻辑；引导补丁则只进行文本编辑，不改变底层代码，包括修改提示词、工具描述和钩子提醒文本。由于这两类补丁可能以不同方式影响支架，我们引入了**分阶段补丁调度**，其作用类似于标准模型训练中的学习率调度。在该调度方案中，优化从能力补丁阶段开始，随后过渡到引导补丁阶段。有关分阶段补丁调度的更多细节见附录 A，诊断与补丁生成指令则见附录 P。

反思会话。 在同一小批量上（并可选择在开发集上）评估打过补丁的支架 H'_n 后，我们进入*反思会话*，从当前支架更新中提取经验。具体而言，*反思智能体*需要比较在小批量上应用补丁前后的智能体性能。得到的结果分为四类：*“已修复”*、*“发生退化”*、*“仍然失败”*和*“仍然通*

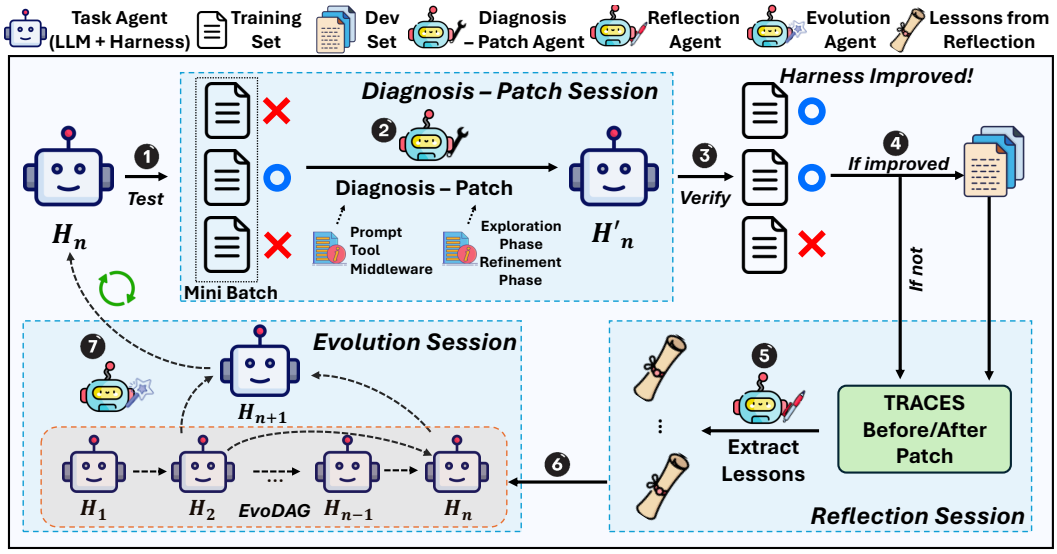


图 2: **AutoSaddler 概览**。迭代优化循环如下：在小批量上测试当前支架，对各个支架组件进行诊断和修补，验证是否取得改进，随后通过反思提取经验并存入 EvoDAG，由其指导下一个支架的演化。

表 1: 采用的补丁类别；各子类型标注为能力补丁（C）或引导补丁（S）。

类别	子类型	说明
提示词补丁	添加提示词规则（S）	在系统提示词中添加或修改行为规则。
	修改提示词规则（S）	修订已有提示词规则，以解决冲突或使指引更加明确。
工具补丁	添加新工具（C）	当现有工具均不支持所需操作时，添加一个新工具。
	修改参数（C）	添加或修复工具参数，以增强筛选或选择能力。
	修复实现（C）	修复错误或扩展工具内部功能，以生成正确结果。
	修复工具描述（S）	修改工具文档字符串，以避免误用工具。
中间件补丁	PreToolUse 钩子（S）	在调用特定工具前即时注入提醒。
	基础设施变更（C）	修改智能体配置、迭代预算或环境设置。
	智能体循环逻辑变更（C）	向智能体循环添加预处理步骤或预算提醒。

过”。针对每个类别，我们提供有针对性的自我反思问题，以引导智能体分析补丁为何有效、解决了哪些失败模式、为何出现退化、补丁为何不足，或补丁是否完全没有产生作用。智能体在反思期间也可以检查轨迹细节。此外，如果补丁触发了开发集上的扩展评估，我们还会引导其反思开发集性能，尤其关注补丁是否以及如何泛化到该小批量之外。这些经验会连同补丁描述、小批量结果和评估指标结果 $\mu(H'_n)$ 一起存储为 EvoDAG 的节点级属性，以促进支架演化。有关反思指令的更多细节见附录 P。

演化会话。 EvoDAG 是一个有向无环图 $\mathcal{G} = (V, E)$，充当优化过程的累积记忆。每个节点 $v_n \in V$ 对应一个先前探索过的支架，并附有相关经验和性能信号。每条有向边 $e \in E$ 表示父支架与其后代支架之间的差异 $\Delta\theta$。

在演化会话中，*演化智能体*查阅完整的 EvoDAG，以合成新的支架 H_{n+1}。智能体不必仅从 H'_n 继续，而是可以在图中所存累积经验的指导下，组合任何一组先前探索过的支架元素。这种合并操作类似于演化搜索，使框架能够通过重组优化历史中不同谱系的成功组件来摆脱局部最优。随后，在下一次迭代中用一个新的小批量评估所得支架 H_{n+1}。有关演化指令的更多细节见附录 P。

设计依据。 总体而言，AutoSaddler 围绕有效自动支架优化的三个要求而设计：*深度诊断*、*结构化干预*和*泛化感知选择*。诊断–补丁会话以对长时程执行轨迹和支架代码库的显式分析为更新依据，从而支持深度诊断，实现深层调试，而非浅层反思。补丁分类体系和分阶段调度将优化限制为针对提示词、工具和中间件的定向更改，而非不受约束的编辑，从而支持结构化干预。反思会话与演化会话结合小批量验证、可选的开发集评估和基于 EvoDAG 的记

表 2: GAIA2 上的测试集 Pass@1 结果，以三次运行的均值 ± 标准差报告。括号中的数值表示任务数量；粗体表示最佳结果。

支架（类型）	GAIA2 世界（Pass@1）			平均值
	21 (107)	22 (112)	27 (81)	
默认智能体（手工）	54.8 ± 4.8	51.5 ± 4.9	52.7 ± 4.3	53.0 ± 1.5
GEPA（自动）	60.1 ± 3.9	47.9 ± 3.4	56.4 ± 0.7	54.6 ± 2.5
Meta-Harness（自动）	53.0 ± 1.1	51.5 ± 5.2	56.0 ± 0.7	53.2 ± 2.2
AutoSaddler（自动）	$\mathbf{61.4 \pm 2.4}$	$\mathbf{60.7 \pm 2.4}$	$\mathbf{64.6 \pm 3.1}$	$\mathbf{62.0 \pm 1.2}$
无深度诊断（自动）	56.7 ± 4.2	57.1 ± 3.9	60.1 ± 5.0	57.8 ± 3.8
无结构化干预（自动）	58.9 ± 2.5	53.3 ± 7.6	59.3 ± 3.3	56.9 ± 3.8
无泛化感知选择（自动）	53.3 ± 1.9	44.9 ± 6.7	54.7 ± 4.0	50.6 ± 4.0

忆机制，从而支持泛化感知选择，使保留的更新更有可能泛化到单条轨迹之外。这些组件共同将支架优化转化为一个迭代过程：诊断故障、应用定向更新，并保留更有可能实现泛化的更改。

5 实验

5.1 实验设置

基准与基础支架系统。 我们在三个各具特点的基准上评估 AutoSaddler：GAIA2 [10]、SWE-Bench Pro（SBP） [9] 和 Terminal-Bench 2.0（TB2） [28]。GAIA2 在模拟智能手机环境中评估通用助手能力，覆盖 10 个不同的*世界*，每个世界都模拟某一角色的日常数字生活。我们使用 GAIA2 提供的默认 ReAct 智能体作为基础支架。SBP 面向取自热门代码仓库的企业级软件工程任务，我们以 SWE-agent [48] 作为基础支架。最后，TB2 包含 89 项现实任务，覆盖系统管理、机器学习和网络安全等领域；对于 TB2，我们采用 Terminus 2 作为基础支架。

实现与基线方法。 我们使用 Claude Agent SDK（CA-SDK） [4] 实现 AutoSaddler 中的三个智能体（即*诊断*–*补丁智能体*、*反思智能体*和*演化智能体*）。详细的智能体指令见附录 P，EvoDAG 的实现细节见附录 B。我们将 AutoSaddler 与 GEPA [1] 和 Meta-Harness [22] 进行比较。GEPA 是一种以提示词为中心、迭代优化系统提示词的基线。相比之下，Meta-Harness 将优化目标从提示词转移到智能体支架，采用精简的系统设计，并使用 CA-SDK 进行端到端系统优化。由于 Meta-Harness 仅在 TB2 上进行过评估，我们将 GEPA 适配到 GAIA2 和 TB2，并将 Meta-Harness 适配到 GAIA2；适配细节见附录 B。

实验规程与数据划分。 除非另有说明，所有优化方法均使用 `Claude Opus 4.6` 作为底层 LLM，并采用默认端点设置。为评估泛化能力，我们确保训练集、开发集和测试集包含来自不同任务组的任务。例如，在 SBP 中，训练集由来自 `qutebrowser` 的任务构成；开发集包含来自 `Vuls` 和 `NodeBB` 的任务；测试集则由来自 `Ansible`、`Flipt` 和 `Element-web` 的任务构成。这一划分使我们能够评估跨代码仓库的泛化能力。三个基准的数据划分详情见附录 B。除非另有说明，对于每种优化方法，我们在训练集和开发集上进行一次演化运行，并在测试集评估时进行三次重复运行。[4]我们以 *Pass@1* 成功率作为评估指标。

5.2 主要结果

我们在表 2 中报告 GAIA2 测试集上的 Pass@1 成功率，并在表 3 中报告 SBP 和 TB2 测试集上的 Pass@1 成功率。可以得到两个主要观察结果。第一，**在三个基准上，AutoSaddler 都能发现比相应基础支架更有效的支架**。具体而言，在 GAIA2 上，AutoSaddler 相比默认智能体提升了 **+9.0 pp**（53.0% → 62.0%）；在 SBP 上，相比 SWE-agent 提升了 **+8.4 pp**（37.3% → 46.9%）；在 TB2 上，相比 Terminus 2 提升了 **+10.0 pp**（40.0% → 50.0%）。第二，**AutoSaddler 的表现优于自动化基线 GEPA 和 Meta-Harness**。与各基准上最强的自动化基线相比，AutoSaddler 在 GAIA2 上提升 **+7.4 pp**（54.6% 与 62.0%），在 SBP 上提升 **+6.2 pp**（42.5% 与 46.9%），在 TB2

[4]由于 Meta-Harness 原生不支持训练集–开发集划分，我们向其提供训练集与开发集的并集，以确保各方法接触的数据量可比。

表 3: SWE-Bench Pro 和 Terminal-Bench 2.0 上的测试集 Pass@1 结果，以三次运行的均值 ± 标准差报告。括号中的数值表示任务数量；粗体表示每列的最佳结果。

SWE-Bench Pro (Pass@1)

支架（类型）	`Ansible` (96)	`Flipt` (85)	`Element-web` (56)	平均值
SWE-agent（手工）	40.6 ± 1.9	31.0 ± 3.5	41.1 ± 1.7	37.3 ± 4.8
GEPA（自动）	50.0 ± 1.3	32.2 ± 1.4	$\mathbf{45.2 \pm 0.6}$	42.5 ± 1.2
Meta-Harness（自动）	36.9 ± 2.9	31.3 ± 1.7	38.7 ± 0.8	35.3 ± 2.0
AutoSaddler（自动）	$\mathbf{58.0 \pm 1.8}$	$\mathbf{36.5 \pm 1.8}$	43.5 ± 1.6	$\mathbf{46.9 \pm 1.8}$

Terminal-Bench 2.0 (Pass@1)

支架（类型）	测试划分（40）
Terminus 2（手工）	40.0 ± 0.0
Terminus KIRA（手工）	47.5 ± 2.5
GEPA（自动）	42.5 ± 2.5
Meta-Harness（自动）	43.3 ± 5.8
AutoSaddler（第 2 次迭代）（自动）	45.0 ± 0.0
AutoSaddler（第 34 次迭代）（自动）	$\mathbf{50.0 \pm 0.0}$

上提升 **+4.4 pp**（43.3% 与 50.0%）。在 TB2 上，AutoSaddler 还以 **+2.5 pp**（47.5% 与 50.0%）的优势超过了由专家手工调优的支架 *Terminus KIRA* [20]。

我们进一步验证了该方法对优化随机性和训练分布偏移的稳健性。在 GAIA2 上独立运行一次 AutoSaddler 可达到 58.6% 的 Pass@1，仍显著高于基础支架及重新运行的基线（附录 C）；而在另一个训练世界上进行优化可达到 57.4%，比基础支架提升 **+5.9 pp**（附录 D）。

除跨任务组评估外，我们还进一步分析了**跨模型迁移能力**。具体而言，我们将 GAIA2 任务智能体的底层 LLM 从 `Opus 4.6` 替换为 `Haiku 4.5`，但保留使用 `Opus 4.6` 优化得到的支架。如附录 E 所示，AutoSaddler 仍比基础支架提升 **+5.6 pp**，表明其可能具备跨模型迁移能力。

为分析优化效率，我们在图 1 中比较了 AutoSaddler 与自动化基线在 GAIA2 上的优化轨迹；附录 J 还给出了 AutoSaddler 完整搜索轨迹的可视化。我们发现，**AutoSaddler 使用显著更少的任务智能体轨迹采样即可取得更高性能**。具体而言，AutoSaddler 仅用 ∼1,000 次轨迹采样便达到 72.3% 的开发集准确率；相比之下，GEPA 和 Meta-Harness 尽管分别消耗了 ∼2,800 次任务执行，性能却饱和于 64.6% 和 61.5%（图 1a）。若以用于学习的轨迹采样数量衡量效率（图 1b），二者对比更加鲜明：AutoSaddler 消耗 147 次轨迹采样后即达到最佳开发集得分，比 Meta-Harness（1,400 次轨迹采样）少约 ∼**10**×。如附录 H 所示，在 TB2 上也观察到类似的效率提升。我们还在附录 I 中进一步刻画了端到端优化成本。尽管 AutoSaddler 每个补丁在优化器侧产生的费用略高，但其选择性评估策略仅用 391 次任务智能体轨迹采样便达到 67.7% 的开发集准确率，已经超过 Meta-Harness 使用 1,400 次轨迹采样后达到的 61.5% 峰值。

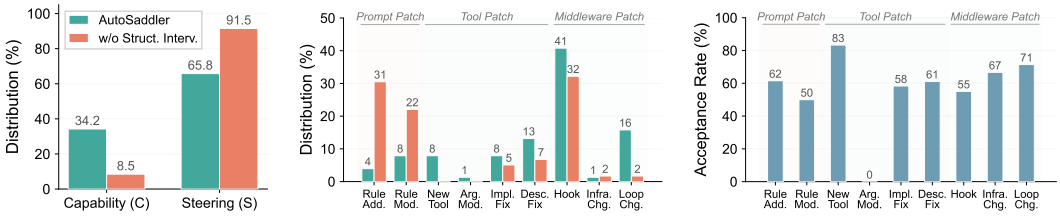
5.3 消融研究与分析

下面给出 GAIA2 上的消融研究，重点考察 AutoSaddler 所依据的三个关键设计原则：*深度诊断*、*结构化干预*和*泛化感知选择*。

RQ1：**深度诊断能否实现更准确的根因识别？** 我们首先开展消融研究，以评估深度诊断能否改善根因识别，并最终促进整体性能提升。具体而言，我们评估一个“*无深度诊断*”变体：用浅层诊断基线替代基于 CA-SDK 的诊断，即通过单次 LLM 调用接收执行轨迹和评估结果，并推断失败原因。这是自动提示词优化流水线在故障反思中常用的策略 [36, 1]。随后将推断出的失败原因传回 CA-SDK，用于生成补丁。相比之下，AutoSaddler 的深度诊断会主动探索执行轨迹和源代码以调查故障。

如表 2 所示，移除深度诊断会显著降低 GAIA2 测试集性能，使 **Pass@1** 从 **62.0** 降至 **57.8**。附录 K 中详细的文件访问和工具调用分析进一步佐证了深度诊断的益处：与仅生成补丁的会话相比，诊断–补丁联合会话在每个优化步骤中平均会多调用 6.2 次工具，并多访问 5.8 次文件。我们还对两种设置下训练期间累计接受的补丁数量进行了定量比较，其中“接受的补丁”是指能在同一小批量上改善性能的补丁；参见图 10，该图位于附录 K。深度诊断始终能产生更多被接受的补丁。到第 1 轮训练结束时（第 25 次迭代），差距已经十分显著（**13** 与 **5**），而且这一优势在整个第 2 轮训练期间（第 26–50 次迭代）得以保持。最后，我们在附录 K 中提供了多个定性案例研究，用以说明浅层诊断与 AutoSaddler 的深度诊断在识别根因方面有何不同。

RQ2：**结构化干预如何影响补丁的多样性和有效性？** 为评估结构化干预的有效性，我们考察一个*无结构化干预*的消融设置，其中移除所提出的补丁分类体系和分阶段补丁调度。该消融设置让智能体在没有明确搜索空间边界的情况下执行不受约束的编辑（与 Meta-Harness 相同）。如表 2 所示，移除结构化补丁机制会显著降低 GAIA2 测试集上的 Pass@1，从 **62.0%**

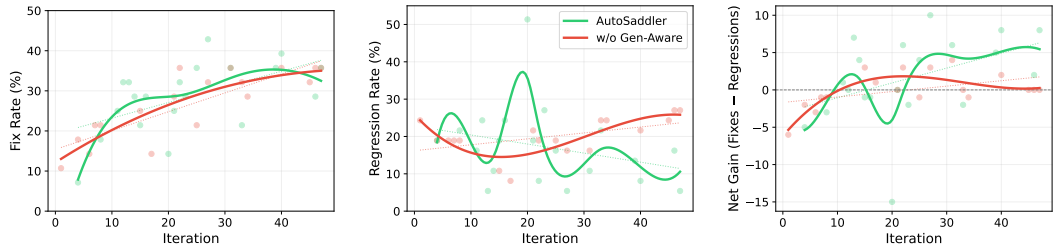


(a) 能力补丁与引导补丁。 (b) 补丁子类型分布。 (c) 各子类型的接受率。

图 3: **补丁类型分布与接受情况：AutoSaddler 与*无结构化干预*。**在没有结构化干预时，补丁高度集中于引导补丁（91.5%）；而 AutoSaddler 会生成涵盖提示词、工具和中间件编辑的均衡组合。

降至 **56.9%**。这一结果表明，将支架优化构建为有明确目标的结构化搜索，对于取得有意义的性能提升至关重要。附录 F 中更细粒度的消融实验进一步表明，仅移除分阶段补丁调度会使 Pass@1 从 60.7% 降至 54.8%，而移除完整的结构化干预则会进一步降至 53.3%。

进一步分析表明，尽管*无结构化干预*设置允许不受约束地编辑补丁，其性能下降仍源于补丁探索存在偏差且补丁多样性降低。如图 3a 和图 3b 所示，在没有结构化干预时，系统会高度集中于引导补丁（91.5%）。这类补丁大多对应直接的文本编辑，系统很少探索基础设施或工具改进等价值更高的干预措施。图 3c 中按补丁子类型分解的接受率进一步说明，不同补丁类型的有效性存在差异。以能力为中心、且不处于提示词层的补丁，即*新增工具*（83%）、*循环变更*（71%）和*基础设施变更*（67%），取得了最高的接受率。然而，在*无结构化干预*设置下，这些关键补丁类型仅占所生成补丁的 **4%**；相比之下，AutoSaddler 通过结构化补丁机制将其占比提升到 **25%** 以上。附录 L 给出了 AutoSaddler 在演化期间发现的这些类别中的代表性补丁。综合来看，这些结果表明，结构化干预会鼓励智能体超越提示词层编辑，探索能力层面的改进。此外，附录 G 表明，能力补丁与引导补丁的修复率相近（55% 与 58%），但前者引发的退化显著更少（8% 与 17%），说明其更新更加持久。



(a) 修复率（基础支架失败）。 (b) 退化率（基础支架通过）。 (c) 开发集净增益。

图 4: **各次迭代在开发集上的性能比较。**AutoSaddler 在取得与*无泛化感知选择*消融设置相近修复率的同时，保持了更低的退化率，因而能持续获得正向净增益，而该消融设置无法维持这种增益。

RQ3：泛化感知选择如何防止过拟合？ 最后，我们开展消融研究，考察泛化感知优化如何缓解对训练集的过拟合，并减少在未见场景上的退化。具体而言，“*无泛化感知选择*”消融设置同时移除反思会话和开发集评估，迫使优化仅依赖训练集上的执行结果。如表 2 所示，移除该组件会显著降低 GAIA2 测试集性能，使 **Pass@1 从 62.0% 降至 50.6%**。这是所有组件消融实验中幅度最大的性能下降。附录 F 中更细粒度的消融实验进一步表明，移除开发集筛选会使 Pass@1 从 60.7% 降至 50.0%，而在此基础上再移除反思机制和 EvoDAG，则会进一步降至 44.9%。

为了量化该机制对开发集表现的影响，图 4 追踪了两种设置在开发集上的表现，并将净性能分解为*修复率*和*退化率*：前者定义为在初始基础支架失败的场景上的成功率，后者定义为在基础支架通过的场景上的失败率。值得注意的是，两种设置的修复率相近（子图 a），这说明性能差距主要并非源自问题求解能力的差异。相反，关键分歧在于退化率（子图 b）。尽管两条曲线在优化期间均有波动，AutoSaddler 的退化率总体呈下降趋势（**-0.24 pp/次迭代**），而消融设置则呈上升趋势（**+0.16 pp/次迭代**）。

我们进一步考察了消融设置在第 21 次迭代时出现的退化率骤增（子图 b，8% → 22%）。在第 20 次迭代中，该消融设置引入了新工具 `send_progress_message_to_user`，并修改了常用工具 `send_message_to_user` 的钩子，强制将智能体重定向到这一新工具。由于没有反思机制评估连带损害，这个范围过宽的补丁被保留下来，干扰了智能体在许多无关开发场景中的行为。有趣的是，同一种失败模式——在高频工具上设置范围过宽、作用超出预期边界的钩子——也曾在 AutoSaddler 的第 4 次迭代中出现（见图 12b，以及附录 M 中的扩展讨论）。然而，得益于 AutoSaddler 的反思机制，此类引发退化的补丁会被阻止。这一直接对比表明，泛化感知选择可以识别并过滤作用范围过宽的补丁，从而帮助维持在未见场景上的性能增益。

最后，图 4c 展示了最终的净增益，表明 AutoSaddler 在第 20 次迭代之后始终领先于消融设置。这说明，AutoSaddler 能够从迭代早期的退化挑战中恢复，并通过从具体优化经验中抽象出可泛化原则，在后续迭代中取得更稳定的改进。有关原则泛化和定性案例研究的更多讨论见附录 M。

6 结论

在本工作中，我们提出了 AutoSaddler，这是一种自动支架优化框架，它将面向 LLM 智能体的支架改进表述为针对执行轨迹的离线学习问题。AutoSaddler 将深入的失败诊断、横跨提示词、工具和中间件的结构化补丁生成，以及通过验证和基于 EvoDAG 的进化实现的关注泛化的更新选择结合起来。因此，它产生的是持久有效的支架更新，而非针对特定轨迹的修补。在 GAIA2、SWE-Bench Pro 和 Terminal-Bench 2.0 上，AutoSaddler 相较对应的基础支架分别稳定提升了 **9.0**、**9.6** 和 **10.0** 个百分点，并且相较各基准上最强的自动化基线分别提升了 **7.4**、**4.4** 和 **6.7** 个百分点。这些结果表明，有效的支架优化需要深度调试、有针对性的干预，以及以泛化能力为目标的显式选择；这也使自动支架优化成为构建性能更强、更可靠的长时程智能体系统的一项重要且前景广阔的研究方向。

7 致谢

我们感谢匿名评审者提出的建设性意见和建议，这些反馈帮助提升了本工作的清晰度与质量。我们感谢 Bo Qiao 在搭建实验所用计算服务器方面给予的慷慨帮助。我们还感谢 GAIA2、SWE-Bench Pro、Terminal-Bench 2.0、SWE-agent、Terminus 2、GEPA 和 Meta-Harness 的创建者与维护者公开提供其基准、系统和工具，从而使本工作的实验得以开展。

参考文献

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA：反思式提示演化可以超越强化学习. In *The Fourteenth International Conference on Learning Representations*, 2026.

[2] Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru WANG, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. 自演化智能体综述：通往人工超级智能之路上演化什么、何时演化、如何演化以及在哪里演化. *Transactions on Machine Learning Research*, 2026. 综述认证.

[3] Anthropic. 面向长时运行智能体的高效支架. `https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents`, 2025. 访问日期：2026-04-21.

[4] Anthropic. Agent SDK 概览：Claude Code 文档. `https://code.claude.com/docs/en/agent-sdk/overview`, 2026. 访问日期：2026-04-21.

[5] Anthropic. 面向长时运行应用开发的支架设计. `https://www.anthropic.com/engineering/harness-design-long-running-apps`, 2026. 访问日期：2026-04-21.

[6] Zhicheng Cai, Xinyuan Guo, Yu Pei, Jiangtao Feng, Jiangjie Chen, Ya-Qin Zhang, Wei-Ying Ma, Mingxuan Wang, and Hao Zhou. FLEX：通过从经验中前向学习实现持续智能体演化. *CoRR*, abs/2511.06449, 2025.

[7] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 从失败轨迹到可靠 LLM 智能体：诊断并修复支架缺陷. *arXiv 预印本 arXiv:2606.06324*, 2026.

[8] Zhengyu Chen, Teng Xiao, Huaisheng Zhu, Yige Yuan, Luan Zhang, and Jingang Wang. Co-Harness：面向 LLM 智能体协同演化支架与模型权重. *arXiv 预印本 arXiv:2607.22688*, 2026.

[9] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro：AI 智能体能否解决长时程软件工程任务？. *CoRR*, abs/2509.16941, 2025.

[10] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Grégoire Mialon, and Thomas Scialom. GAIA2：在动态异步环境中评测 LLM 智能体. *CoRR*, abs/2602.11964, 2026.

[11] Jia Fu, Xiaoting Qin, Fangkai Yang, Lu Wang, Jue Zhang, Qingwei Lin, Yubo Chen, Dongmei Zhang, Saravan Rajmohan, and Qi Zhang. AutoRAG-HP：面向检索增强生成的自动在线超参数调优. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, Findings of ACL, pages 3875–3891. Association for Computational Linguistics, 2024.

[12] Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. 谁引入了失败？通过频谱分析自动归因多智能体系统的失败. *arXiv 预印本 arXiv:2509.13782*, 2025.

[13] Hanghui Guo, Weijie Shi, Zhangze Chen, Shengxiang Xu, Yishu Wang, Yimei Zhang, Wangze Ni, Jia Zhu, and Shimin Di. DREvo：蒸馏经重新校准的历史经验以实现支架自演化. *arXiv 预印本 arXiv:2607.26722*, 2026.

[14] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 结合大语言模型与演化算法可产生强大的提示优化器. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[15] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA：通过支架与权重更新实现自我改进的 AI. *arXiv 预印本 arXiv:2605.27276*, 2026.

[16] Shengran Hu, Cong Lu, and Jeff Clune. 智能体系统的自动化设计. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[17] Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. MemoHarness：从经验中学习的智能体支架. *arXiv 预印本 arXiv:2607.14159*, 2026.

[18] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy：将声明式语言模型调用编译为最先进的流水线. In *The Twelfth International Conference on Learning Representations*, 2024.

[19] Shashank Kirtania, Param Biyani, Priyanshu Gupta, Yasharth Bajpai, Roshni Iyer, Sumit Gulwani, and Gustavo Soares. 通过 BREW 改进语言智能体. *CoRR*, abs/2511.20297, 2025.

[20] KRAFTON AI and Ludo Robotics. Terminus-KIRA：以最小支架提升前沿模型在 Terminal-Bench 上的性能. https://github.com/krafton-ai/kira, 2026.

[21] LangChain. 借助支架工程改进深度智能体. https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering, 2026. 访问日期：2026-04-21.

[22] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness：模型支架的端到端优化. *arXiv 预印本 arXiv:2603.28052*, 2026.

[23] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. 智能体支架工程：可观测性驱动的编程智能体支架自动演化. *arXiv 预印本 arXiv:2604.25850*, 2026.

[24] Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z Pan, et al. EvoX：面向自动发现的元演化. *arXiv 预印本 arXiv:2602.23413*, 2026.

[25] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P Murphy. AutoHarness：通过自动合成代码支架改进 LLM 智能体. *arXiv 预印本 arXiv:2603.03329*, 2026.

[26] Guoqing Ma, Jia Zhu, Hanghui Guo, Weijie Shi, Jiawei Shen, Jingjiang Liu, and Yidan Liang. 基于因果推断的多智能体系统自动失败归因与关键步骤预测方法. *arXiv 预印本 arXiv:2509.08682*, 2025.

[27] Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. DoVer：面向 LLM 多智能体系统的干预驱动自动调试. In *The Fourteenth International Conference on Learning Representations*, 2026.

[28] Mike A. Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, Estefany Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis-Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-Bench：在命令行界面的高难度现实任务上评测智能体. *CoRR*, abs/2601.11868, 2026.

[29] Nous Research. Hermes Agent：与你共同成长的智能体. https://hermes-agent.nousresearch.com/, 2026. 访问日期：2026-04-21.

[30] Alexander Novikov, Ngân Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve：用于科学与算法发现的编程智能体. *CoRR*, abs/2506.13131, 2025.

[31] OpenAI. 支架工程：在智能体优先的世界中利用 Codex. https://openai.com/index/harness-engineering/, 2026. 访问日期：2026-04-21.

[32] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 面向多阶段语言模型程序的指令与示例优化. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 9340–9366. Association for Computational Linguistics, 2024.

[33] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank：利用推理记忆扩展智能体自演化. In *The Fourteenth International Conference on Learning Representations*, 2026.

[34] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. 自然语言智能体支架. *arXiv 预印本 arXiv:2603.25723*, 2026.

[35] Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. 在黑暗中演化智能体：通过自偏好进行回溯式支架优化. *arXiv 预印本 arXiv:2606.05922*, 2026.

[36] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 基于"梯度下降"和束搜索的自动提示优化. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 7957–7968. Association for Computational Linguistics, 2023.

[37] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita：以最少预定义和最大程度自演化实现可扩展智能体推理的通用智能体. *CoRR*, abs/2505.20286, 2025.

[38] Biswa Sengupta and Jinhua Wang. Harbor：自动化支架优化. *arXiv 预印本 arXiv:2604.20938*, 2026.

[39] Asankhaya Sharma. OpenEvolve：一个开源演化式编程智能体. `https://github.com/algorithmicsuperintelligence/openevolve`, 2025. 访问日期：2026-04-21.

[40] Varun Ursekar, Apaar Shanker, Veronica Chatrath, Sam Denton, et al. VeRO：用于智能体优化智能体的评测支架. *arXiv 预印本 arXiv:2602.22480*, 2026.

[41] Chi Wang, Xueqing Liu, and Ahmed Hassan Awadallah. 面向大语言模型生成推理的高性价比超参数优化. In Aleksandra Faust, Roman Garnett, Colin White, Frank Hutter, and Jacob R. Gardner, editors, *International Conference on Automated Machine Learning, 12-15 November 2023, Hasso Plattner Institute, Potsdam, Germany*, Proceedings of Machine Learning Research, pages 21/1–17. PMLR, 2023.

[42] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-G\"odel Machine：通过近似最优自改进机器开发人类水平的编程智能体. In *The Fourteenth International Conference on Learning Representations*, 2026.

[43] Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, Nebojsa Jojic, Eric P. Xing, and Zhiting Hu. PromptAgent：使用语言模型进行策略规划，实现专家级提示优化. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[44] Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX：用于演化智能体工作流的自动化框架. In Ivan Habernal, Peter Schulam, and Jörg Tiedemann, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025 - System Demonstrations, Suzhou, China, November 4-9, 2025*, pages 643–655. Association for Computational Linguistics, 2025.

[45] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent：软件工程智能体能否实时自演化？. *CoRR*, abs/2511.13646, 2025.

[46] Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Noveen Sachdeva, Isabella Ye, Weili Wang, et al. PACEvolve：实现长时程进度感知的一致演化. *arXiv 预印本 arXiv:2601.10657*, 2026.

[47] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 大语言模型即优化器. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[48] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent：智能体-计算机接口赋能自动化软件工程. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[49] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent：用于递归自我改进的自指智能体框架. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 27890–27913. Association for Computational Linguistics, 2025.

[50] Mert Yüksekgönül, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad：通过文本实现自动"微分". *Nature*, 639, 2025.

[51] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer：谁在 LLM 智能体系统中引发失败？. *CoRR*, abs/2509.03312, 2025.

[52] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness：能够自我改进的支架. *arXiv 预印本 arXiv:2606.09498*, 2026.

[53] Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. GraphTracer：面向稳健多轮深度搜索的图引导 LLM 智能体失败追踪. *arXiv 预印本 arXiv:2510.10581*, 2025.

[54] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin Gödel Machine：自改进智能体的开放式演化. In *The Fourteenth International Conference on Learning Representations*, 2026.

[55] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob N. Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *CoRR*, abs/2603.19461, 2026.

[56] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow：自动生成智能体工作流. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[57] Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, et al. HarnessCompass：引导自动支架演化，获得可泛化且有效的智能体支架. *arXiv 预印本 arXiv:2608.01918*, 2026.

[58] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 哪个智能体在何时导致任务失败？论 LLM 多智能体系统的自动失败归因. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.

[59] Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. 将函数作为可学习权重的语言模型智能体离线训练. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 60315–60335. PMLR / OpenReview.net, 2024.

[60] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL：LLM 智能体是经验学习者. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth*

Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada, pages 19632–19642. AAAI Press, 2024.

[61] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, et al. LLM 智能体中的外部化：记忆、技能、协议与支架工程的统一综述. *arXiv 预印本 arXiv:2604.08224*, 2026.

[62] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 大语言模型是人类水平的提示工程师. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[63] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, et al. LLM 智能体在何处失败，以及如何从失败中学习. *arXiv 预印本 arXiv:2509.25370*, 2025.

[64] Runchuan Zhu, Bowen Jiang, Lingrui Mei, Fangkai Yang, Lu Wang, Haoxiang Gao, Fengshuo Bai, Pu Zhao, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AdaptFlow：通过元学习实现自适应工作流优化. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 3287–3302. Association for Computational Linguistics, 2025.

传统小批量训练	AutoSaddler 阶段	解释
步骤 1：采样小批量	❶ 采样用于评估当前支架的小批量	作用相同
步骤 2：执行前向传播	❶ 执行当前支架	智能体执行即前向计算；它同时产生最终答案和一条轨迹。
步骤 3：计算损失	❶ 记录结果和轨迹	任务结果提供稀疏的指标信号；轨迹提供解释该结果所需的证据。
步骤 4：反向传播梯度	❷ 诊断并打补丁 + ❸ 在同一小批量上验证	AutoSaddler 没有数值梯度；它通过针对根因、干预和验证生成假设来构造文本更新。
步骤 6：验证/保存检查点	❹ 在开发集上评估	开发集评估检查候选改进能否泛化到该小批量之外。
步骤 5：应用优化器更新	❺ 反思 + ❻ 存入 EvoDAG + ❼ 演化下一个支架	提交的更新不只是原始补丁。AutoSaddler 会更新优化器状态，并据此合成下一个候选支架。
步骤 7：重复	下一次迭代	在一个新的小批量上评估演化得到的支架。

表 4: 传统小批量训练与图 2 中 AutoSaddler 循环之间的对应关系。AutoSaddler 遵循小批量学习的外层结构，但用基于轨迹的符号优化取代了可微梯度计算。

A 将 AutoSaddler 视为文本化支架参数上的小批量学习

本附录阐明图 2 中的 AutoSaddler 循环与标准小批量训练循环之间的关系。这一类比很有帮助，因为 AutoSaddler 保留了小批量学习的外层结构：它反复在一个小批量上评估候选系统、提取更新信号、检查该更新能否泛化，并利用累积的优化历史提出下一个候选方案。不过，不应从字面上理解这一类比。与神经网络参数不同，支架参数是文本化、可执行且不可微的。因此，AutoSaddler 用基于轨迹的符号优化过程取代了可微梯度计算。

与传统小批量训练的对应关系。 表 4 总结了这种对应关系。我们将传统小批量训练的各阶段记为步骤 1–步骤 7，并使用图 2 中的带圈标号指代 AutoSaddler 的各阶段。

为何反向传播对应于诊断、打补丁和验证。 这两种设定之间最核心的不匹配之处，在于反向传播的对应物。在神经网络训练中，一旦计算出损失，反向传播就会给出数学上明确定义的梯度，因此更新方向相对明确。

在 AutoSaddler 中，一次失败的智能体执行并不会直接指明应当如何修改支架。根因可能位于提示词、工具接口、工具实现、中间件钩子或智能体循环中。因此，系统必须从轨迹中推断根因假设，实施有针对性的干预，并通过实验检验该干预能否改善同一小批量上的行为：

$$\text{diagnosis} \longrightarrow \text{patch-as-intervention} \longrightarrow \text{same-batch verification}. \tag{2}$$

这一序列承担了构造可信文本梯度的作用。需要特别强调的是，此阶段的补丁尚不宜被视为最终提交的参数更新；它更像是用来检验所推断根因是否合理的一次干预。这一区分很重要，因为数值梯度由固定的数学算子导出，而文本梯度则依赖推断、具有语义性且可能出错。因此，同批次验证是必要的，可避免把未经检验的解释当作可靠的更新方向。

为何优化器更新由反思与 EvoDAG 演化共同构成。 第二处不匹配涉及优化器更新这一概念。在常规训练中，优化器直接修改数值参数。而在 AutoSaddler 中，原始补丁只是一个候选变更。在允许该变更影响后续搜索之前，系统必须判断它在当前小批量之外是否仍然有效，以及应当保留哪些可复用经验（如果存在）。

因此，已提交的更新更适合被理解为

$$\text{dev-set validation} \longrightarrow \text{reflection} \longrightarrow \text{EvoDAG update} \longrightarrow \text{evolution of the next harness}. \tag{3}$$

反思将变更前后的证据提炼为可复用经验，EvoDAG 则将这些经验与补丁描述及性能信号一并存储。随后，演化会话利用这一符号优化器状态，在构造下一个候选方案时，从先前探索过的支架中选择、修订或重组组件。从这个意义上说，EvoDAG 是一种优化器记忆。类似

算法 1 将 AutoSaddler 视为支架参数上的小批量学习

Require: Initial harness H_0, training set D_{train}, development set D_{dev}, rollout budget K
Initialize EvoDAG G_0 with H_0
for iteration $n = 0, 1, 2, \ldots$ until budget K is exhausted **do**
  Sample mini-batch $B_n \subset D_{\text{train}}$ ▷ 图 2: ❶; 步骤 1
  Execute H_n on B_n, collecting outcomes and traces τ_n ▷ ❶; 步骤 2–3
  Diagnose failures in τ_n and infer root-cause hypotheses ▷ ❷; 文本信用分配
  Generate targeted patch $\Delta\theta_n$, yielding candidate $H'_n = H_n + \Delta\theta_n$ ▷ ❷; 作为干预的补丁
  Re-evaluate H'_n on B_n ▷ ❸; 验证文本梯度
  if same-batch performance improves **then**
    Evaluate H'_n on D_{dev} ▷ ❹; 步骤 6 泛化门禁
  end if
  Reflect on before/after traces and evaluation results ▷ ❺; 将证据转化为经验
  Update EvoDAG G_n with patch, metrics, and lessons to obtain G_{n+1} ▷ ❻; 符号优化器状态
  Evolve next harness H_{n+1} from G_{n+1} ▷ ❼; 步骤 5 提交更新
end for
return best harness selected by development performance

于动量通过累积过去的梯度信息来平滑噪声更新，并将未来步骤引向历史上更有效的方向，EvoDAG 会累积过去的补丁、结果与反思，引导支架朝着已有证据表明可产生泛化收益的变更演化。因此，AutoSaddler 的更新既能利用历史又具有组合性，而不是一次局部算术运算。

关于顺序的说明。 上述对应关系有意交换了步骤 5 与步骤 6 的通常顺序。在标准小批量训练中，通常先应用优化器更新，再进行验证或保存检查点。在 AutoSaddler 中，开发集验证先于最终的符号优化器更新，因为 EvoDAG 是优化器状态的一部分。一旦存入误导性的经验，它就可能影响后续演化。因此，在系统将经验提交到 EvoDAG 并演化 H_{n+1} 之前，验证充当一道泛化门禁。

算法视角。 算法 1 以算法形式呈现了同一视角。关键在于，❷–❸ 中生成的补丁充当一次干预，用于检验推断出的文本梯度。只有在验证、反思以及基于 EvoDAG 的演化之后，搜索轨迹才会发生最终更新。

分阶段补丁调度。 我们根据补丁对支架所作变更的类型，将表 1 中的补丁分为两类。**能力补丁**修改可执行代码或编排逻辑，包括工具实现、工具参数、基础设施设置和智能体循环逻辑。这类补丁能够改变智能体可执行的动作，或支架执行这些动作的方式。**引导补丁**是保持底层可执行代码不变的文本编辑，包括修改提示词、工具描述和钩子提醒文本。这类补丁主要用于细化智能体如何从既有能力中做出选择，以及如何遵循特定任务的约束。

这一区分与基于梯度的优化中的大、小学习率步长相似，但并不完全等同。能力补丁往往表现得像较大步长，因为它们可以引入新功能、改变控制流或调整可用动作空间。引导补丁往往表现得像较小步长，因为它们是在既有能力集合内调整智能体行为。不过，这种类比只是近似的：提示词或钩子编辑有时也会造成较大的行为变化，尤其是在影响高频决策时。因此，我们将能力–引导区分用作调度启发式，而不是对影响大小的严格保证，并依靠验证和开发集评估来检测范围过宽或会引发回归的变更。

受这一类比启发，AutoSaddler 采用以 k 为转折点的两阶段调度：

1. **探索阶段**（$n \le k$）。搜索优先采用能力补丁，以解决工具、基础设施和智能体循环行为中的根本缺口。
2. **精炼阶段**（$n > k$）。当能力集合稳定后，搜索切换至引导补丁以细化行为。

转折点 k 可以直接指定为迭代次数，也可以通过训练轮数间接指定：

$$k = E \cdot \left\lceil \frac{|\mathcal{D}_{\text{train}}|}{B} \right\rceil,$$

其中，E 是能力阶段的训练轮数，B 是小批量大小。与学习率调度类似，若将 k 设得过小，可能无法充分探索高影响力的能力改进；若设得过大，则可能延后风险较低的行为精炼。在我们的实验中，一个能力阶段训练轮（$E = 1$）效果良好：搜索先使用高影响力补丁遍历一次训练集，再切换到有针对性的引导调整。

要点。 AutoSaddler 可以被视为在外置智能体参数上进行的小批量离线学习。它的设计不同于标准梯度训练，因为支架参数是文本化且不可微的；性能信号稀疏且延迟；长时程轨迹上的信用分配也存在歧义。因此，这些额外机制并非无关紧要。轨迹收集、诊断、将补丁作为干预、同批次验证、开发集验证、反思、EvoDAG 记忆与演化，正是传统小批量训练中由损失函数、反向传播、优化器状态和检查点机制紧凑完成之工作的符号化对应物。

B 实验设置的更多细节

EvoDAG 的实现。 EvoDAG 底层的 DAG 很容易实现，但要让智能体高效访问该 DAG 则更具挑战：随着候选方案和轨迹在迭代过程中不断累积，直接浏览 EvoDAG 会变得不切实际。我们没有将完整 DAG 序列化进提示词，而是引入 `evo-dag` 命令行接口（CLI），以便 CA-SDK 与 EvoDAG 交互。该接口按需提供 DAG 的结构化摘要，包括补丁历史、已学习经验、场景注册表和支架代码差异。这样，智能体可以先识别相关上下文，再深入具体原始轨迹或源文件进行详细分析。完整命令集汇总于 表 5。

表 5: `evo-dag` CLI 提供的命令：按用途分为读取指定范围 EvoDAG 视图的读操作，以及在每个会话中更新节点或场景的写操作。

命令	用途	使用时机
读操作		
`evo-dag summary`	DAG 拓扑、最佳候选方案、全部边	会话开始时；快速了解全局
`evo-dag show history`	完整补丁历史：差异、反思、经验	深入查看先前尝试
`evo-dag show node <idx>`	节点详情：分数、意图、裁决、输出目录	检查特定候选方案
`evo-dag show edge <parent> <child>`	代码差异、逐场景影响、变更文件	理解特定补丁
`evo-dag show scenario <id>`	逐场景历史、根因、已尝试的修复	诊断失败场景之前
`evo-dag show current-batch`	当前小批量的场景 ID 与输出目录	了解当前迭代
`evo-dag show lineage`	带边类型的 DAG 谱系	理解分支结构
`evo-dag show lessons`	累积的良好/不良模式	打补丁前检查已知模式
写操作		
`evo-dag update-selection`	记录选定的父候选方案及选择理由	演化会话中，选择父节点之后
`evo-dag update-intent`	记录目标场景、诊断、方法、变更文件和变更摘要	诊断–补丁会话中，应用补丁之后
`evo-dag update-reflection`	记录逐场景状态（`fixed` / `regressed` / `still_failing` / `still_passing`）、根因、补丁后解释、预防措施或下一步，以及泛化说明	反思会话中，逐场景记录

将基线适配到我们的基准。 GEPA 将系统提示词视为一个统一字符串。相比之下，GAIA2 中默认的 ReAct 智能体将其系统提示词分散在多个部分，例如核心行为原则、ReAct JSON 工具调用格式，以及 Meta 的 Agents Research Environments（ARE）模拟环境说明。这些部分被定义为独立变量，仅在推理时拼接起来，由此产生的结构不匹配使 GEPA 无法直接应用。为解决这一问题，我们将各部分拼接成一个字符串，使用 `===== VAR_NAME =====` 分隔符标记各部分边界，并要求 GEPA 的反思 LLM 在提示词演化期间保留这些分隔符。随后，演化后的提示词沿分隔符拆分，所得各部分传给 CA-SDK，以更新对应变量。对于 SBP 和 TB2，我们分别采用 SWE-agent 与 Terminus 2 作为基础支架；由于两个智能体都将其系统提示词定义为一个统一字符串，因此 GEPA 无需进一步修改即可直接应用。

原始 Meta-Harness 假设支架包含在单个 Python 脚本中，并要求继承固定的智能体基类。然而，在 GAIA2 默认智能体中，支架可能分布于整个代码仓库。因此，我们将 Meta-Harness 的适配范围扩展至仓库级别，要求 CA-SDK 在整个仓库中应用与原始 Meta-Harness 相同的补丁流程。这使得支架优化能够超越单文件设定，实现整体优化。

各基准的专用评估协议。 对于每个基准，我们都遵循其官方评估基础设施，以确保所报告结果可与先前排行榜数值直接比较。

GAIA2。我们通过 ARE 使用 `are-run` 命令运行默认的 ReAct 智能体；该命令在对应的沙箱化世界中执行每项任务，并记录完整轨迹。遵循 GAIA2 协议 [10]，我们使用 `Llama-3.3-70B-Instruct` 作为裁判模型，根据智能体最终状态判断任务是否成功。

SWE-Bench Pro。遵循 [9] 的官方协议，每个由智能体生成的补丁都会应用于逐实例 Docker 镜像内，该镜像已预先配置好处于问题对应提交版本的代码仓库。随后，使用该实例的 *fail-to-pass* 与 *pass-to-pass* 单元测试验证补丁。只有当所有 fail-to-pass 测试通过且没有 pass-to-pass 测试发生回归时，任务才被视为已解决。

Terminal-Bench 2.0。遵循 [28]，我们使用 `Harbor` 在隔离的 Docker 容器中运行每项任务。任务是否成功由专用测试脚本判定，该脚本检查最终的终端与系统状态。

各基准的数据划分。 为评估优化后的支架能否泛化到优化期间未见过的任务，我们分别为每个基准构建训练集、开发集和测试集，汇总见 表 6。对于具有自然分组结构的基准，我们按任务组而不是按单个任务划分。在 GAIA2 中，划分轴为世界，每个世界对应一种不同的人设和模拟数字环境。这一设计确保测试集包含来自优化期间未见任务组的任务，从而比随机的任务级划分更有力地衡量分布外泛化。在 SWE-Bench Pro 中，划分轴为代码仓库，这也会带来编程语言、代码库结构和问题分布的变化。该设计同样确保测试集包含来自优化期间未见任务组的任务，从而比随机任务级划分更有力地衡量分布外泛化。Terminal-Bench 2.0 的任务横跨多个领域，但没有自然的分组轴；因此，我们将其均匀随机划分为训练集、开发集和测试集。

表 6: 三个基准的数据划分。为评估分布偏移下的泛化能力，我们对每个基准进行划分，使训练集、开发集和测试集中的任务来自*互不重叠的任务组*，而不是采用随机的任务级划分：GAIA2 按世界（人设）划分，SWE-Bench Pro 按代码仓库（编程语言）划分。Terminal-Bench 2.0 仅包含 89 个跨多个领域的任务，且没有自然的分组轴，因此采用均匀随机划分。

基准	划分轴	划分	任务组	任务数
GAIA2	世界（人设）	训练集	**世界** `29`	75
		开发集	**世界** `30`	65
		测试集	**世界** `21`	107
			世界 `22`	112
			世界 `27`	81
SWE-Bench Pro	代码仓库（语言）	训练集	`qutebrowser`（Python）	79
		开发集	`Vuls`（Go）	40
			`NodeBB`（JavaScript）	40
		测试集	`Ansible`（Python）	96
			`Flipt`（Go）	85
			`Element-web`（TypeScript）	56
Terminal-Bench 2.0	随机 †	训练集	—	30
		开发集	—	19
		测试集	—	40

† 仅有 89 个任务，分布于异构领域（系统管理、机器学习、网络安全）；不存在可用于构造分布偏移划分的自然轴。

优化预算。 对于 AutoSaddler 和 GEPA，我们在 GAIA2 与 SWE-Bench-Pro 上优化 2 个训练轮，在 Terminal-Bench 2.0 上优化 4 个训练轮；TB2 的预算翻倍，是为了补偿其训练集规模仅约为 GAIA2 的一半。对于 Meta-Harness，采用相同的按训练轮计预算会造成误导：AutoSaddler 和 GEPA 执行小批量优化，而 Meta-Harness 采用全批量模式，因此 2 个训练轮

只对应两次补丁更新，会使该基线受到不公平限制。为确保比较公平，我们改为按照训练期间消耗的任务执行总次数来匹配优化预算。具体而言，我们在 GAIA2 上训练 Meta-Harness 20 个训练轮，在 Terminal-Bench 2.0 上训练 15 个训练轮，并在 SWE-Bench-Pro 上训练 8 个训练轮，从而确保其在每个基准上的任务执行总次数均不少于 AutoSaddler。

C 对优化随机性的稳健性

表 7: **对优化随机性的稳健性**。两次独立优化运行所得支架在 GAIA2 世界 22 上的 Pass@1。测试性能报告为三次执行的均值 ± 标准差。

方法	**GAIA2 世界 22（112 个场景；Pass@1）**
默认智能体（人工）	51.5 ± 4.9
GEPA（运行 1）	47.9 ± 3.4
GEPA（运行 2）	50.6 ± 0.5
Meta-Harness（运行 1）	51.5 ± 5.2
Meta-Harness（运行 2）	51.2 ± 2.2
AutoSaddler（运行 1）	$\mathbf{60.7 \pm 2.4}$
AutoSaddler（运行 2）	58.6 ± 0.5

由于端到端支架优化的成本显著高于重复进行测试时评估，我们的主实验对每种方法仅执行一次优化。为评估观察到的增益是否对随机优化轨迹敏感，我们在 GAIA2 上针对 AutoSaddler、GEPA 和 Meta-Harness 各额外执行一次独立优化。我们在世界 22 上评估每个所得支架；该世界是最大的留出测试世界，包含 112 个场景，每个支架重复执行三次。

表 7 报告了结果。AutoSaddler 的第二次优化运行取得 58.6% 的 Pass@1，仅比第一次运行低 2.1 个百分点，并比默认智能体高 7.1 个百分点。它还分别比独立重跑所得的 GEPA 和 Meta-Harness 支架高 8.0 和 7.4 个百分点。尽管两条优化轨迹不足以对优化器进行全面的统计刻画，但独立运行中一致的增益表明，AutoSaddler 的改进并非某一条有利搜索轨迹所特有。

D 对训练分布偏移的稳健性

表 8: **对训练分布偏移的稳健性**。使用不同训练世界独立优化 AutoSaddler 时，其在 GAIA2 世界 22 上的 Pass@1。数值为三次测试执行的均值 ± 标准差。

方法	**GAIA2 世界 22（112 个场景；Pass@1）**
默认智能体（人工）	51.5 ± 4.9
AutoSaddler（运行 1；世界 29）	$\mathbf{60.7 \pm 2.4}$
AutoSaddler（运行 2；世界 29）	58.6 ± 0.5
AutoSaddler（不同训练集；世界 24）	57.4 ± 2.1

GAIA2 主实验使用世界 29 作为训练集来优化支架。为考察所得增益是否关键依赖于这一特定训练分布，我们用规模相近的世界 24 替换世界 29，在保持其余优化和评估协议不变的情况下独立优化 AutoSaddler。所得支架在世界 22 上评估三次。

如表 8 所示，在世界 24 上优化的支架取得 57.4% 的 Pass@1。尽管使用了不同的训练世界进行优化，它仍比默认智能体高 5.9 个百分点，且与在世界 29 上训练的第二次独立 AutoSaddler 运行相差不超过 1.2 个百分点。这些结果表明，AutoSaddler 能从不同的人设和任务分布中发现有效的支架更新，而不是依赖原始世界 29 训练划分所特有的属性。

E 向较弱智能体骨干模型的跨模型迁移能力

表 2 报告了以 `Claude Opus 4.6` 同时作为优化器和智能体骨干模型时的结果，但一个关键的实际问题是：由强模型优化的支架在部署于较弱任务智能体时能否保持收益。为检验这一点，我们使用 `Claude Haiku 4.5` 作为任务智能体骨干模型，在 GAIA2 上重新评估所有方法，同时保持基于 Opus 优化所得的支架不变。表 9 表明，AutoSaddler 相较默认智能体取

得了 +5.6 pp 的整体提升，展现出有效的跨模型迁移能力。值得注意的是，在所有被评估的世界上，AutoSaddler 始终优于全部基线方法和消融设定。

表 9: **跨模型迁移能力**。以 `Claude Haiku 4.5` 作为智能体骨干模型、使用由 `Claude Opus 4.6` 优化的支架时，GAIA2 测试集上的 Pass@1 成功率。数值为三次运行的均值 ± 标准差。

支架（类型）	GAIA2 世界（Pass@1）			平均值
	21 (107)	**22 (112)**	**27 (81)**	
默认智能体（人工）	31.2 ± 5.9	24.4 ± 1.4	36.2 ± 1.9	30.0 ± 1.8
GEPA（自动）	33.6 ± 1.9	24.4 ± 0.5	38.3 ± 1.2	31.4 ± 0.4
Meta-Harness（自动）	30.8 ± 0.0	26.8 ± 4.7	34.2 ± 4.0	30.2 ± 2.7
AutoSaddler（自动）	$\mathbf{38.3 \pm 0.0}$	$\mathbf{30.4 \pm 1.8}$	$\mathbf{39.1 \pm 2.6}$	$\mathbf{35.6 \pm 0.8}$
无深度诊断（自动）	36.1 ± 5.5	19.0 ± 5.2	34.2 ± 0.7	29.2 ± 3.4
无结构化干预（自动）	32.1 ± 3.0	28.0 ± 4.0	37.4 ± 5.8	32.0 ± 3.5
无泛化感知选择（自动）	28.3 ± 2.9	23.2 ± 4.1	30.0 ± 3.8	26.9 ± 1.3

F 结构化干预与泛化感知选择的细粒度消融

表 10: **细粒度消融**。GAIA2 世界 22 上的 Pass@1。数值为三次运行的均值 ± 标准差。

方法	GAIA2 世界 22（112 个场景；Pass@1）
默认智能体（人工）	51.5 ± 4.9
AutoSaddler	$\mathbf{60.7 \pm 2.4}$
无结构化干预	53.3 ± 7.6
无阶段调度	54.8 ± 4.4
无泛化感知选择	44.9 ± 6.7
无开发集过滤	50.0 ± 5.0

主消融实验分别整体移除 AutoSaddler 的每项设计原则。我们进一步拆解*结构化干预*和*泛化感知选择*背后的各个机制。对于结构化干预，主要的*无结构化干预*消融同时移除了补丁分类体系和分阶段补丁调度。因此，我们引入更细粒度的*无阶段调度*变体：保留补丁分类体系，仅移除分阶段调度。类似地，主要的*无泛化感知选择*消融同时移除了开发集过滤以及结合 EvoDAG 的反思。我们引入*无开发集过滤*变体：保留结合 EvoDAG 的反思，但移除开发集过滤。对于每个变体，我们都重新运行完整优化流程，并在 GAIA2 世界 22 上通过三次重复测试执行评估所得支架。

表 10 展示了结果。仅移除分阶段补丁调度会使 Pass@1 从 60.7% 降至 54.8%，下降 5.9 个百分点；进一步移除补丁分类体系会使性能继续降至 53.3%，表明对补丁空间进行结构化还能额外贡献 1.5 个百分点。对于泛化感知选择，移除开发集过滤会使 Pass@1 从 60.7% 降至 50.0%，而进一步移除结合 EvoDAG 的反思会使性能继续降至 44.9%。因此，在这一比较中，开发集过滤的影响更大，而结合 EvoDAG 的反思额外带来 5.1 个百分点的增益。总体而言，这些细粒度消融表明，每项设计原则中的两个机制都对最终支架性能有贡献。

G 补丁持久性分析

理想的支架更新不应只修复触发该补丁的场景，还应保持原本已经正确的行为。因此，我们通过比较同一训练小批量上每个补丁生成前后的任务结果，分析补丁的持久性。我们将*修复率*定义为应用补丁后由失败转为通过的场景占先前失败场景的比例，将*回归率*定义为应用补丁后由通过转为失败的场景占先前通过场景的比例。

首先，我们按照补丁分类体系中的三个支架组件对补丁分组：提示词、工具和中间件。如表 11 所示，三类补丁的修复率相近，均为 57–59%；工具补丁的回归率略高，为 19%，而提示词和中间件补丁均为 14%。因此，仅凭所编辑的组件无法看出清晰的持久性模式。

这些类别之间区分度较弱，一个原因是它们混合了不同的干预机制。例如，工具补丁既包括新工具和实现修复等可执行变更，也包括对工具描述的文本变更。类似地，中间件补丁既包

表 11: **按支架组件划分的补丁持久性**。修复率和回归率通过比较同一训练小批量上补丁前后的结果来衡量。

补丁类别	修复率（%）	回归率（%）
提示词	59	14
工具	57	19
中间件	57	14

括可执行的基础设施或智能体循环变更，也包括纯文本的 PreToolUse 钩子。因此，我们使用正文中更高层级的“能力–引导”分类体系对所有生成的补丁重新分类。能力补丁修改可执行功能或编排逻辑，而引导补丁改变文本指令但不改变底层可执行能力。

表 12 显示出更清晰的差异。引导补丁的修复率略高于能力补丁（58% 对 55%），但其回归率超过后者的两倍（17% 对 8%）。因此，能力补丁实现了相近的局部修复率，同时将回归率降低 9 个百分点。这一结果表明，能力层面的干预往往产生更持久的更新，而文本引导更容易产生超出预期范围的溢出影响。这一量化模式也与我们的定性示例一致：范围明确的可执行修复通常解决确定性的能力限制，而范围过宽的钩子或提示词规则可能影响无关场景。

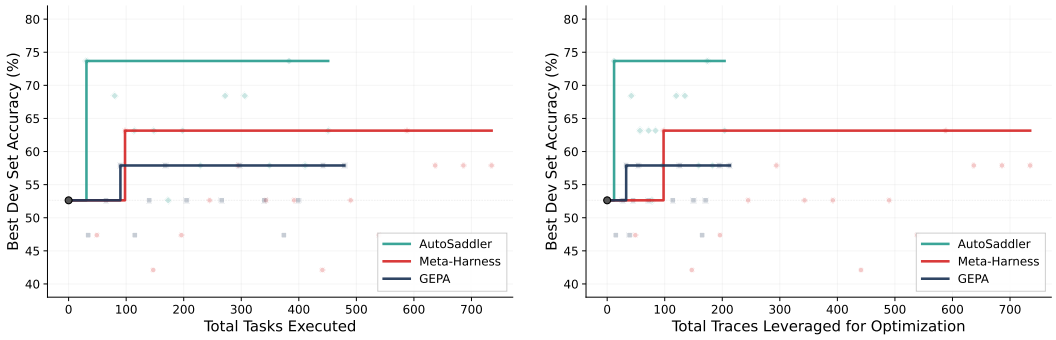
表 12: **按干预机制划分的补丁持久性**。能力补丁与引导补丁的修复率相近，但引导补丁会引发多得多的回归。

补丁类型	修复率（%）	回归率（%）
引导补丁	58	17
能力补丁	55	8

H Terminal-Bench 2.0 上的优化效率

为评估 AutoSaddler 的优化效率能否泛化到 GAIA2 之外，我们报告其在 Terminal-Bench 2.0 上的计算效率与学习效率；该基准面向一种不同的智能体能力，即长时程命令行问题求解。我们沿用 GAIA2 主实验（第 5 节）的协议，将 AutoSaddler 与 Meta-Harness 和 GEPA 比较。

图 5 报告了 Terminal-Bench 2.0 上的计算与学习效率。从共同的 52.6% 起点出发，AutoSaddler 仅执行 31 次任务并利用 12 条轨迹后，开发集准确率便达到 **73.7%**，比 Meta-Harness（63.2%）高 10.5 个百分点，比 GEPA（57.9%）高 15.8 个百分点。相较 Meta-Harness（98 条轨迹），所利用轨迹数减少约 ∼8×，这与 GAIA2 上观察到的趋势一致，表明在以终端为中心、失败主要由工具使用和环境交互错误而非多跳推理导致的任务上，AutoSaddler 的诊断引导式补丁也能带来一致的优化效率增益。



(a) 计算效率　　(b) 学习效率

图 5: Terminal-Bench 2.0 上的优化性能与效率比较。**(a)** AutoSaddler 仅执行 31 次任务便达到 73.7% 的开发集准确率，而 Meta-Harness 需要 98 次执行才稳定在 63.2%，GEPA 在 90 次执行时达到 57.9%。**(b)** 按优化所利用的执行轨迹数衡量，AutoSaddler 仅利用 12 条轨迹便取得最佳性能，比 Meta-Harness（98 条轨迹）少 8× 以上，同时高出 10.5 个百分点。

I 端到端优化成本刻画

仅凭任务智能体执行并不能完整刻画自动支架优化的计算成本。除支架评估外，每个优化器还可能因候选方案生成、诊断、打补丁、反思和候选方案选择而产生大量优化器侧 LLM 开销。因此，我们分别分析 GAIA2 上的（i）优化器侧 LLM 开销和（ii）任务智能体评估成本。

优化器侧成本。 表 13 报告了生成、拒绝和接受的补丁数量，以及每个生成补丁的平均墙钟时间、货币成本、LLM 调用次数和词元用量。GEPA 的优化器侧成本最低，每个补丁为 $5.50，但其搜索空间仅限系统提示词。Meta-Harness 和 AutoSaddler 则优化范围更广的支架组件，因此二者更适合直接比较。AutoSaddler 每生成一个补丁的成本为 $14.56，比 Meta-Harness 高 $1.91；但每个补丁仅需 533 秒，而非 883 秒，墙钟时间降低 39.6%。尽管 AutoSaddler 显式执行诊断、结构化补丁生成、反思和演化，这一差异仍然成立。

表 13: **GAIA2 上的优化器侧成本。**运行时间、货币成本、LLM 调用次数和词元用量均按每个生成补丁取平均。

方法	生成的补丁	拒绝的补丁	接受的补丁	墙钟时间（秒）/补丁	成本（$）/补丁	LLM 调用/补丁	输出词元/补丁	缓存创建输入词元/补丁	缓存读取输入词元/补丁
GEPA	76	58	18	386	5.50	21.3	11,416	156,788	280,940
Meta-Harness[5]	20	–	–	883	12.65	66.9	43,429	250,466	3,528,484
AutoSaddler	39	19	20	533	14.56	76.1	21,464	430,933	3,246,158

任务智能体评估成本。 支架评估的成本远高于上述优化器侧操作。如表 14 所示，单次 GAIA2 任务智能体执行平均需要 20.2 次 LLM 调用、550,988 个输入词元、7,380 个输出词元和 203.9 秒墙钟时间。

表 14: **每次 GAIA2 执行的平均任务智能体评估成本。**

指标	每次执行的平均值
LLM 调用次数	20.2
输入词元	550,988
输出词元	7,380
墙钟时间	203.9 s

各方法的评估协议在调用这一高成本操作的频率上差异显著。Meta-Harness 会对每个候选更新评估全部 140 个 GAIA2 训练与开发场景的并集。相比之下，AutoSaddler 每次优化迭代评估六个训练场景，并且仅对改善训练小批量的补丁执行包含 65 个场景的开发集评估。因此，AutoSaddler 在 391 次任务智能体执行后便达到 67.7% 的开发集准确率，已经超过 Meta-Harness 在 1,400 次执行后达到的 61.5% 峰值。由此可见，虽然 AutoSaddler 每生成一个补丁的优化器侧货币成本略高，但它更有选择地使用高成本任务智能体评估，并在小得多的执行预算下取得更高性能，足以抵消这部分开销。

J 搜索轨迹可视化

为说明反思会话和演化会话如何共同塑造搜索轨迹，图 6 可视化了 AutoSaddler 在 GAIA2 上 50 次迭代（2 个训练轮）期间开发集分数的变化。每个节点表示一个在小批量上被接受、随后在留出开发集上评估的支架（总计 51 个候选方案中的 21 个）；图中显示的分数为开发集准确率。实线边表示基础继承（细灰线表示顺序继承，粗深色线表示变基），红色虚线边表示拣选合并；正是这些操作形成了 DAG 结构，而非简单链条。

该轨迹呈现四个阶段。在*基础构建*阶段（第 0–8 次迭代），演化会话在每一步都选择唯一可用的前驱，从而形成一条线性链。在*快速改进*阶段（第 8–15 次迭代），连续五次接受的更新线性叠加，并在第 13 次迭代达到 67.7%。DAG 结构在*选择性合并与修复*阶段（第 16–27 次迭代）形成：第 20 次迭代时，高频工具上的一个钩子导致性能灾难性回归至 33.8%；随后演化会话变基到第 13 次迭代，并将第 13 和第 14 次迭代中经过验证的修复拣选到第 21 与第 22 次迭代，同时撤销反思所识别出的有害变更。第 27 次迭代通过线性链继承这些精选

[5]Meta-Harness 未采用显式的补丁接受门禁，因此拒绝和接受的补丁数量不适用。

补丁，达到 72.3% 的全局峰值。在*巩固与变基*阶段（第 31–47 次迭代），所有分支均变基到第 27 次迭代。第 46 次迭代过度累积了从第 40、44 和 45 次迭代拣选的八个钩子和规则，导致下降 −12.3 pp；第 47 次迭代的演化会话将其诊断为集体引导开销，并裁剪到仅保留四个保守补丁，使性能恢复至 69.2%。总体而言，该轨迹突显了 AutoSaddler 如何在支架搜索期间利用反思引导的变基与选择性合并，保留有益变更、舍弃有害变更并从回归中恢复。

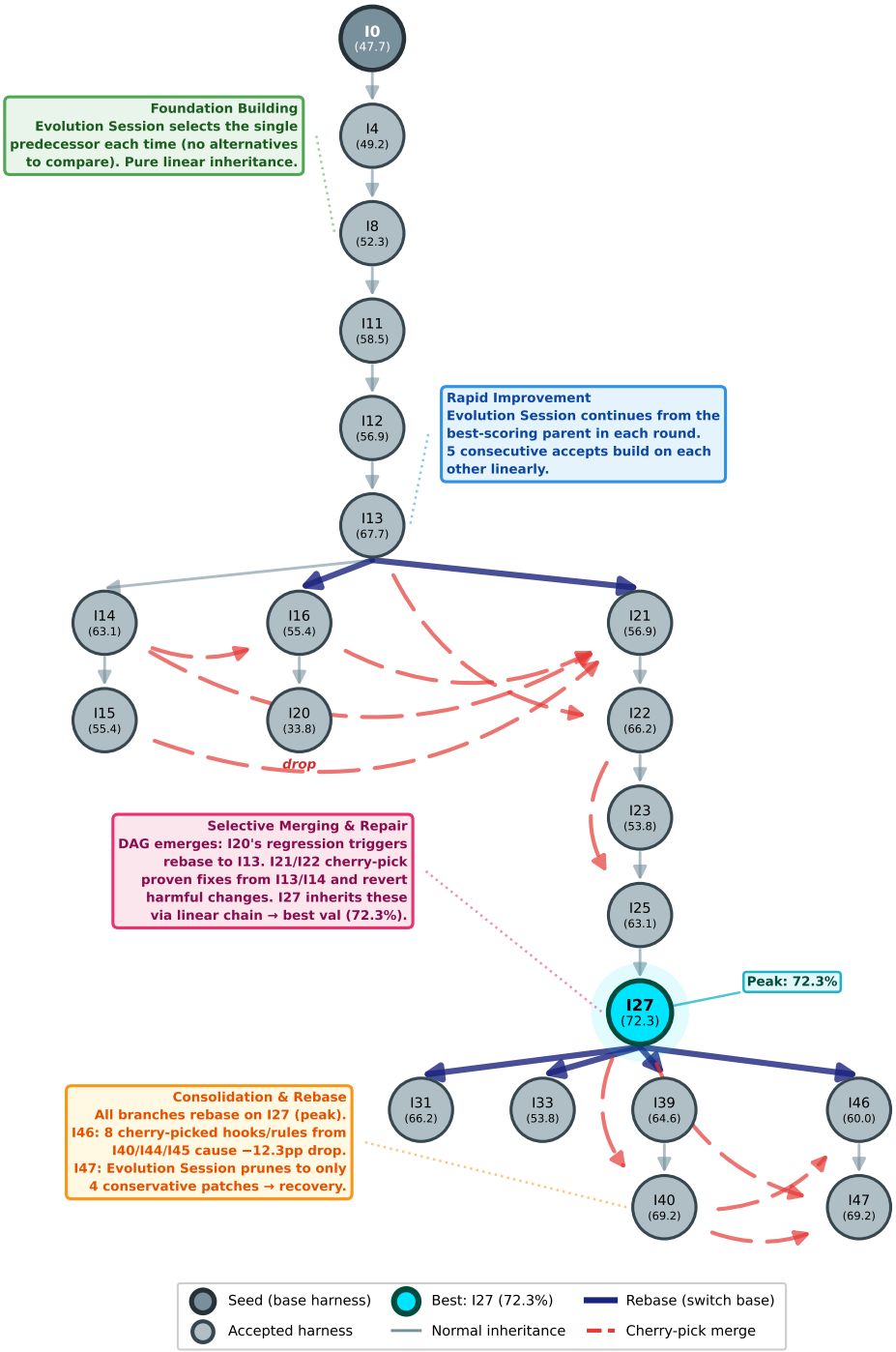


图 6: AutoSaddler 以 EvoDAG 表示的演化搜索轨迹。四个阶段（基础构建、快速改进、选择性合并和巩固）使其在第 27 次迭代达到 72.3% 的开发集全局峰值，摆脱了第 20 次迭代的回归和局部最优。

K RQ1 的补充分析

深度诊断的工具调用与文件访问开销 AutoSaddler 的一项核心设计选择是深度诊断阶段，其目的在于比“*无深度诊断*”消融设定中的浅层反思更彻底地调查失败轨迹。为验证该阶段确实进行了实质上更深入的分析，而非仅仅增加表面开销，我们通过实验测量 AutoSaddler 相较消融设定额外消耗的工具调用和文件访问次数。在 AutoSaddler 中，Claude Agent SDK 同时负责诊断和打补丁；而在消融变体中，SDK 只负责打补丁。因此，两者之间的差距直接反映深度诊断带来的调查工作量。

工具调用很容易计数，因为 Claude Agent SDK 的每次工具调用都会被记录。然而，准确统计文件访问并非易事，因为 Claude Agent SDK 偶尔会将工作委派给子智能体，而子智能体内部发出的工具调用不会记录在主轨迹中。为确保两个变体的测量口径一致，我们按以下标准统计主智能体的文件访问：

- **通过专用文件工具访问文件**。统计 Claude Agent SDK 的专用文件处理工具调用次数，即 `Read`、`Write`、`Edit`、`MultiEdit` 和 `Grep`。
- **通过 Bash 命令访问文件**。统计读取或操作文件内容的 Shell 命令执行次数，包括 `cat`、`wc`、`grep`，以及会打开文件的 Python 调用（例如包含 `open()` 调用的 `python3` 或 `python` 脚本）。

我们对 AutoSaddler 和消融变体应用相同的统计规则，从而可以直接比较调查工作量。

表 15 报告了 AutoSaddler 和“*无深度诊断*”消融设定每个优化步骤的平均工具调用与文件访问次数。AutoSaddler 平均执行 69.7 次工具调用和 45.5 次文件访问，而消融变体分别为 63.5 次和 39.7 次，即每步多出 6.2 次工具调用和 5.8 次文件访问。

表 15: AutoSaddler 及其无深度诊断消融变体每个优化步骤的平均工具调用和文件访问次数。

方法	平均工具调用次数	平均文件访问次数
无深度诊断	63.5	39.7
AutoSaddler	69.7	45.5
Δ（归因于深度诊断）	+6.2	+5.8

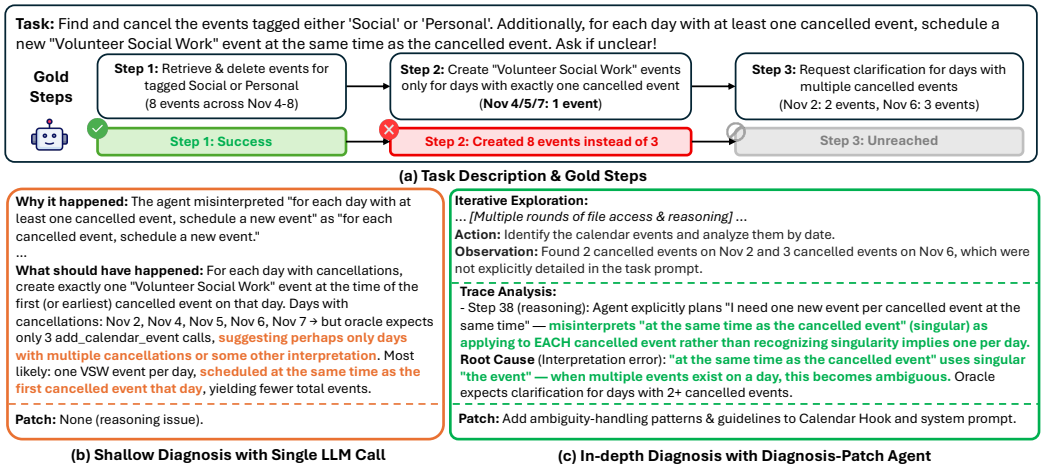


图 7: 一个取消操作存在歧义的日历任务案例。与浅层诊断不同，AutoSaddler 的深度诊断通过深入调试准确识别出真正根因。

定性案例研究。 图 7 展示了这种深度如何转化为诊断准确性。在该任务中，智能体应仅在只有一次取消操作的日期（11 月 4、5、7 日）创建新日程，并对存在多次取消操作的日期（11 月 2、6 日）请求澄清。可是，它忽略了任务歧义，为所有被取消的日程都创建了新日程。浅层诊断没有进行彻底调查，错误地假设智能体只是误解了指令，并认为在存在多次取消操作的日期，智能体应匹配第一次取消的时间。因此，它将该问题误归类为简单的 LLM

推理错误，并跳过了打补丁。相比之下，AutoSaddler 识别日历日程并按日期分析，确认 11 月 2 日和 6 日存在多次取消操作，同时与预期日程交叉验证。由此，它精准定位到真正根因：单数短语“该日程”的语言歧义。它通过在日历钩子和系统提示词中加入歧义处理准则，成功解决了该问题。

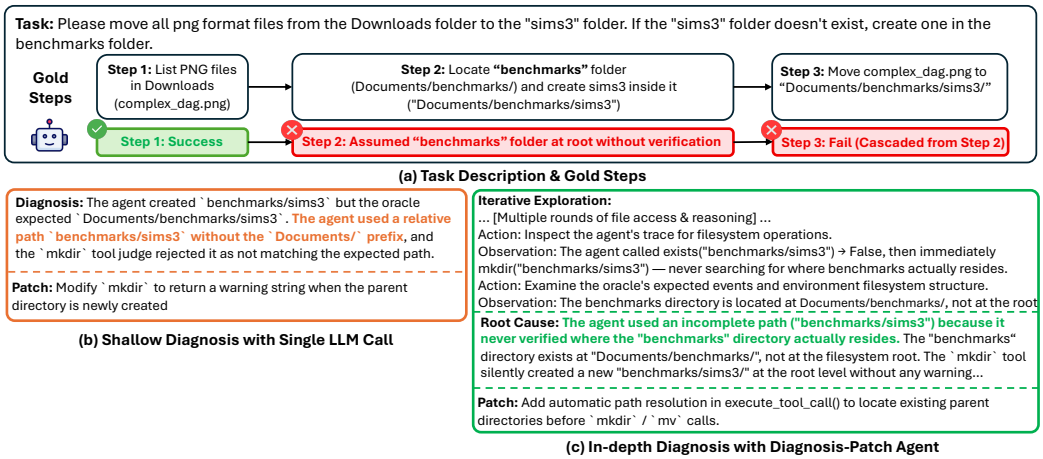


图 8: 一个文件管理任务案例：要求将 PNG 文件移入 `benchmarks` 下的 `sims3` 文件夹。浅层诊断将失败错误归因于相对路径问题，而 AutoSaddler 验证文件系统并定位到真正原因：未验证父目录。

图 8 展示了一种互补的失败模式：浅层反思给出了听起来合理、但事实错误的诊断。在该任务中，智能体本应先定位 `Documents/` 下已有的 `benchmarks` 目录，再在其中创建 `sims3`。但智能体未验证 `benchmarks` 的实际位置，直接在文件系统根目录创建了 `benchmarks/sims3/`。浅层反思将失败诊断为：智能体明知 benchmarks 位于 `Documents/` 下，却使用了错误的*相对路径*；这与智能体根本没有验证过位置的实际行为不一致。因此，它提出一个被动补丁，只修改 `mkdir`，让其在新建父目录时发出警告，却没有解决智能体缺乏事前验证的问题。相比之下，AutoSaddler 检查智能体的文件系统轨迹，发现智能体调用 `exists("benchmarks/sims3")` 后立即对同一路径调用 `mkdir`，并未搜索真实位置；随后又与环境文件系统结构交叉验证，确认 `benchmarks` 位于 `Documents/benchmarks/`。由此，它精准定位到真正根因：在调用 `mkdir` 和 `mv` 前没有验证父目录。它通过为 `execute_tool_call()` 增加自动路径解析成功解决了问题，从源头消除了静默错放。

图 9 展示了一个失败源于智能体从未观察到某个候选项的案例。在该任务中，智能体本应检测到 9 月 30 日从火车站出发的两个候选行程，并请用户澄清目的地。可是，智能体只依据第一个匹配行程就确信无疑地预订了一辆厢式车，忽略了索引 19 处的第二个候选项。浅层反思没有检查完整行程历史，因而未能识别索引 19 处存在第二个匹配行程，反而围绕联系人识别和预期调用次数进行诊断，错过了真正的失败原因。因此，它提出的修复没有处理被遗漏的候选项。相比之下，AutoSaddler 直接检查出租车应用环境状态中的完整行程历史，并按日期过滤，发现 9 月 30 日有两个行程：索引 5 处的 Ahmedabad Railway Station，以及索引 19 处的 Chennai Central Railway Station；它还将此与预期的零次 `order_ride` 调用交叉验证。由此，它精准定位到真正根因：智能体遗漏了索引 19 处的匹配行程，因而未能发现目的地歧义。它通过增加 `search_ride_history` 工具成功解决了问题；该工具可用日期、出发地和目的地关键词过滤搜索行程历史，在一次调用中呈现相互冲突的候选项，并触发正确的澄清行为。

已接受补丁的累积。 图 10 可视化了训练迭代过程中已接受补丁的累积数量，对正文中的讨论形成补充。结果显示，AutoSaddler 与消融变体之间的差距在训练早期、即第 25 次迭代之前便已出现，并持续贯穿第 2 个训练轮。这证实深度诊断组件会带来更多被接受的补丁，并最终取得更好性能。

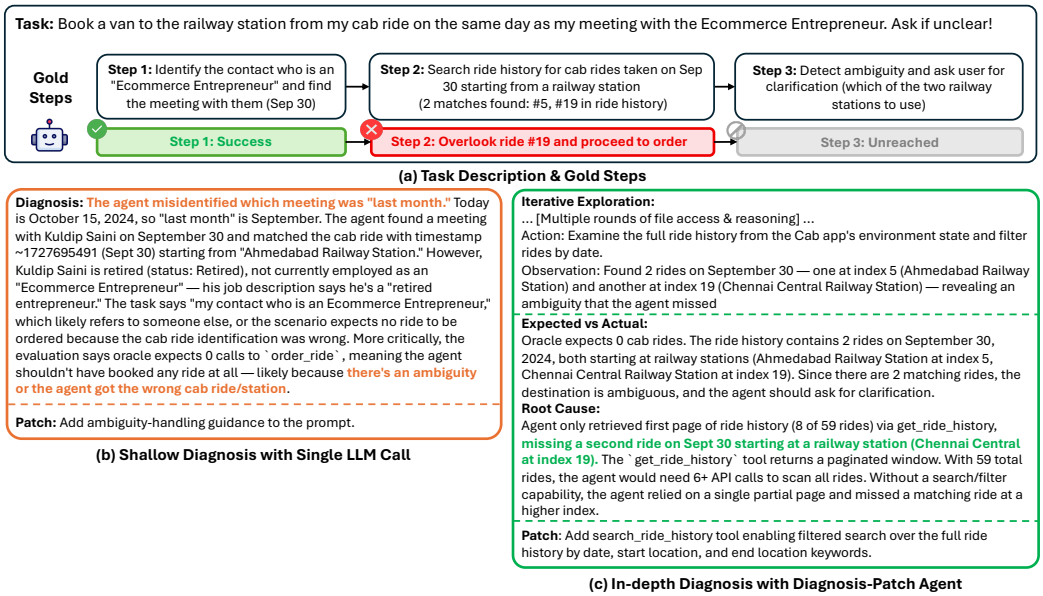


图 9: 一个同一天存在两个候选行程的出租车预订任务案例。浅层诊断忽略了第二个匹配行程并提出偏离目标的修复，而 AutoSaddler 检查完整行程历史，识别出被遗漏的候选项。

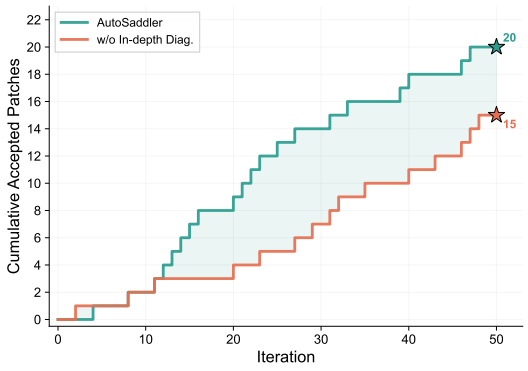


图 10: 已接受补丁的累积数量。 AutoSaddler 累积了 20 个已接受补丁，而无深度诊断消融变体累积了 15 个。

L RQ2 的补充分析

图 11 中的开发集优化轨迹展示了结构化打补丁与分阶段补丁调度背后的具体机制。在探索阶段（第 1–25 次迭代），搜索优先采用能力补丁，解决无法单靠提示词编辑消除的结构性限制。例如，在第 11 次迭代，AutoSaddler 引入批量 API `get_crime_rates_batch`，将多次串行查询合并，大幅减少步骤预算消耗。在第 13 次迭代，AutoSaddler 识别出环境轮询逻辑中的执行停滞，并通过实现基于通知的退出策略解决该问题。

当这些能力层面的改进使支架稳定后，精炼阶段（第 26–50 次迭代）将搜索转向引导补丁，以微调智能体行为。例如，在第 27 次迭代，AutoSaddler 用建设性的回复模板替换了禁止性钩子，使先前因过度纠正而失败的案例恢复成功，并达到 72.3% 的开发集准确率峰值。总体而言，该轨迹表明，结构化打补丁与分阶段补丁调度先扩展支架的功能能力，再细化其决策行为，从而带来 24.6 *pp* 的累积提升。

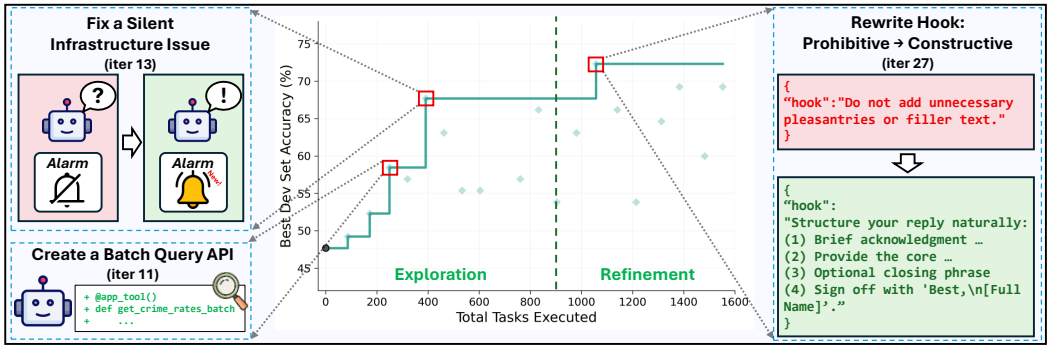


图 11: 优化轨迹中发现的代表性补丁。在探索阶段（第 1–25 次迭代），搜索以能力补丁为主，包括新增工具、实现修复和智能体循环逻辑变更，用于解决支架中的结构性限制（如第 11 和 13 次迭代）。在精炼阶段（第 26–50 次迭代），搜索转向引导补丁，以细化提示词规则、工具描述和 PreToolUse 钩子（如第 27 次迭代）。两个阶段合计带来 24.6 *pp* 的累积提升。

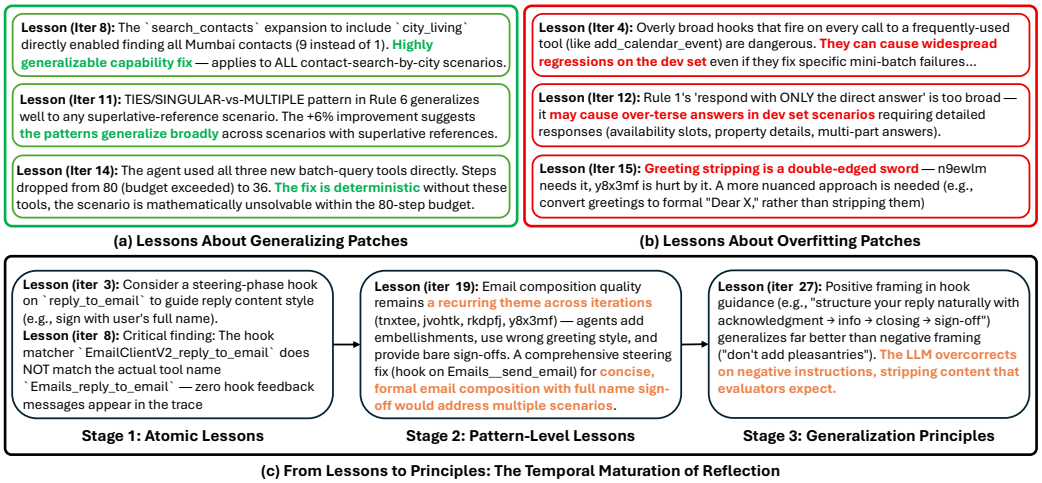


图 12: 反思从（a）可泛化的补丁，以及（b）虽然小批量结果相同但发生过拟合的补丁中提取经验。（c）与 `reply_to_email` 相关的失败谱系展示了原子经验如何逐步演化为一条泛化原则。

M RQ3 的补充分析

我们通过检查每次迭代中反思所提取的经验（图 12），进一步研究 AutoSaddler 与“*无泛化感知选择*”消融设定之间回归差距背后的机制。由此得到两个关键观察。第一，子图 (a) 和 (b) 表明，反思会话通过联合分析开发集分数波动和补丁内容，能够准确区分可泛化补丁与过拟合补丁。例如，反思正确认可覆盖一致场景类别的补丁，如针对基于城市查询的搜索字段扩展（第 8 次迭代），或针对最高级指代的平局消解规则（第 11 次迭代）。反之，它也能成功标记副作用溢出到非预期场景的过拟合补丁，例如对所有日程创建操作无差别触发的日历钩子（第 4 次迭代），或破坏需要结构化细节之场景的简洁性规则（第 12 次迭代）。这些累积判断为泛化感知选择提供了关键过滤信号。第二，子图 (c) 追踪“邮件回复质量”这一问题在四次迭代中的变化，展示了反思对该问题的认识如何改变形式：从场景特定的诊断（第 3 次迭代识别出需要回复钩子；第 8 次迭代捕获匹配器缺陷），到跨四个场景识别出的模式（第 19 次迭代），最终形成关于应如何表述引导文本、与场景无关的原则（第 27 次迭代）。成熟的不是问题本身，而是反思对该问题的抽象层次；这体现了累积学习，而非反复重新发现。

N GAIA2 上发现的补丁目录

表 16 展示了*基础支架*（优化前的种子支架）与 AutoSaddler 在 GAIA2 上训练后产生的*最终支架* $\hat{\theta}_{\text{AS}}$ 之间的代表性补丁。补丁按照表 1 中的分类体系，按类别和子类型分组；插入的文本以蓝绿色标示，基础支架条目若为斜体，则表示对应组件或功能在补丁之前并不存在。

表 16: AutoSaddler 在 GAIA2 上发现的代表性补丁；这些补丁将基础支架转化为最终支架 $\hat{\theta}_{\text{AS}}$。C = 能力，S = 引导。插入的文本以蓝绿色标示。

C/S	组件	基础支架	最终支架（$\hat{\theta}_{\text{AS}}$）
提示词补丁	（*对 `default_agent/prompts/system_prompt.py` 的修改*）		
	提示词规则修改		
S	规则 1（沟通）	1. COMMUNICATION: Only message the user when completely done or if the task is impossible.	1. COMMUNICATION: Only message the user when completely done or if the task is impossible. When the user asks a factual question expecting a specific answer (a city name, a number, a person's name, etc.), respond with ONLY the direct answer — no extra calculations, context, or formatting.
S	规则 6（歧义）	6. AMBIGUITY: Execute all clear and unambiguous parts of a request immediately. When you encounter ambiguities, contradictions, or impossible elements, finish unambiguous subtasks and then stop and explicitly ask the user for clarification before proceeding with those specific parts.	6. AMBIGUITY: Execute all clear and unambiguous parts … ask the user for clarification before proceeding with those specific parts. A subtask is CLEAR if all its required tool parameters (recipients, content, IDs, paths) can be fully determined — even if it semantically relates to an ambiguous part. Common ambiguity patterns that REQUIRE clarification: (a) TIES — when the task references 'the item with the highest/lowest X' but multiple items are tied at that value; (b) SINGULAR vs MULTIPLE — when the task uses singular language ('the event', 'the property') but multiple items match the criteria; (c) PER-GROUP vs PER-ITEM — when applying an action 'for each day/group' at a time derived from a singular source, but multiple sources exist within that group.
	新增提示词规则		
S	规则 7（任务分解）	*(rule does not exist)*	7. TASK DECOMPOSITION: When the user's request contains multiple ordered steps (signaled by words like "then", "after that", "next", "afterwards"), treat each step as a separate subtask. Complete each subtask fully before moving to the next. If a later subtask has ambiguities but earlier subtasks are clear, execute all clear subtasks first, then ask about the ambiguous ones. CRITICAL: Ordering words create HARD DEPENDENCY BOUNDARIES — subtasks after the boundary MUST NOT be executed until all preceding subtasks are completed, even if the later subtask's parameters are fully determinable.
工具补丁			
	新增工具		

（下页续）

（表 16 接上页）

C/S	组件	基础支架	最终支架（$\hat{\theta}_{\text{AS}}$）
C	`CityApp.get_crime_rates_batch`	*(tool does not exist)*	`get_crime_rates_batch(zip_codes: list[str]) -> dict` Batch crime-rate lookup over many zip codes in a single call; replaces N serial queries that exhausted the step budget.
C	`EmailClientApp.get_email_addresses_in_folder`	*(tool does not exist)*	`get_email_addresses_in_folder(folder_name: str = "INBOX") -> dict` Returns all unique sender/recipient addresses in a folder for fast contact discovery.
C	`MessagingApp.get_all_participants`	*(tool does not exist)*	`get_all_participants() -> dict` Returns all unique participant names across conversations, excluding "Me".
C	`MessagingAppV2.get_all_participants`	*(tool does not exist)*	`get_all_participants() -> dict` V2 analog: returns unique participant IDs, excluding the current user.
C	`CabApp.search_ride_history`	*(tool does not exist; only paginated `get_ride_history` was available)*	`search_ride_history(date: str | None = None, start_location_keyword: str | None = None, end_location_keyword: str | None = None) -> dict` Searches the full ride history with optional filters in one call.
	实现修复		
C	`ContactsApp.search_contacts`	Searches `first_name`, `last_name`, `phone_number`, `email` only.	Additionally searches `city_living`, `country`, `nationality`, `job`, `address`, enabling city- and role-based contact lookup.
	中间件补丁		
	PreToolUse 钩子		
S	`Emails__reply_to_email` (reminder text in `hook.json`)	"Do not add unnecessary pleasantries or filler text." (prohibitive: triggers over-correction — agent strips content evaluators expect.)	"Structure your reply naturally: (1) brief acknowledgment, (2) provide the core information, (3) optional closing phrase, (4) sign off with 'Best,\n[Full Name]'." (constructive: provides a positive structural template.)
	智能体循环逻辑变更		
C	`are_simulation_main.py` (agent loop)	The benchmark environment does not notify the agent of cab status changes (e.g., delays). Without a mechanism to advance simulated time between turns, the agent loop stalls indefinitely, unable to reach the point where it could proactively discover these state changes.	AutoSaddler resolves the stall by inserting an inter-turn time-advancement call: when no messages are pending, the loop invokes `wait_for_notification(timeout=300)` to fast-forward simulated time by up to 300 seconds. If no notification arrives, the timeout itself wakes the agent, prompting it to proactively poll for state changes.
C	`_resolve_fs_path` in `execute_tool_call()` on `*__mkdir` (`path`), `*__mv` (`path2`)	`path="benchmarks/sims3"` (no automatic resolution: `mkdir` silently creates a new `benchmarks/sims3/` at the filesystem root, despite an existing `Documents/benchmarks/` elsewhere.)	`path="Documents/benchmarks/sims3"` (automatic path resolution: locates existing parent directories with the same basename via filesystem search before `mkdir`/`mv` is invoked.)

O Terminal-Bench 2.0 上发现的补丁目录

表 17 展示了*基础支架*（优化前的种子支架）与 AutoSaddler 在 Terminal-Bench 2.0 上训练后产生的*最终支架* $\hat{\theta}_{\text{AS}}$ 之间的代表性补丁。补丁按照 表 1 中的分类体系，按类别和子类型分组；插入的文本以蓝绿色标示，基础支架条目若为斜体，则表示对应组件或功能在补丁之前并不存在。

表 17: AutoSaddler 在 Terminal-Bench 2.0 上发现的代表性补丁；这些补丁将基础支架转化为最终支架 $\hat{\theta}_{\text{AS}}$。C = 能力，S = 引导。插入的文本以蓝绿色标示。

C/S	组件	基础支架	最终支架（$\hat{\theta}_{\text{AS}}$）
提示词补丁	（*对系统提示词模板* `prompt-templates/verify-before-complete.txt` *的修改*）		
	新增提示词规则		
S	完成前验证	*(no verification guidance; the base prompt ends immediately after the format specification)*	CRITICAL – Verification before completion: You MUST verify your solution works before setting `"task_complete": true`. When setting up services, test endpoints. When creating files, verify contents. Verify file permissions for all service users. Perform a dry run of automated hooks. If verification fails, fix and verify again before completing.
S	不同输入测试与构建产物清理	*(rule does not exist)*	For scripts or programs, test with a DIFFERENT input than the provided example to confirm generalizability. After testing, clean up the output directory: remove any build artifacts (compiled binaries, `.o` files), temporary files, or intermediate outputs that the task did NOT ask you to create. Verifiers often check that output directories contain ONLY the expected files.
S	库偏好	*(rule does not exist)*	When writing scripts, check the terminal output for pre-installed packages and prefer established libraries over fragile custom implementations.
S	HTML 净化	*(rule does not exist)*	For HTML sanitization / XSS filtering, ALWAYS use BeautifulSoup (not regex). Decompose dangerous tags (`script`, `noscript`, `iframe`, `object`, `embed`, `frame`, `style`) and do a string replacement to strip `javascript:` from entire output as final safety pass.
工具补丁			
	参数修改		
C	`_limit_output_length` (`max_bytes` parameter)	`_limit_output_length(output, max_bytes=10000)` Default 10 KB limit truncates large file contents or command outputs, causing the agent to miss critical information.	`_limit_output_length(output, max_bytes=30000)` 30 KB limit provides more room while keeping context manageable.
	实现修复		
C	`_get_prompt_template_path` (prompt template selection)	Returns the **default** Terminus-2 prompt template path. The default prompt contains only JSON format specification without domain-specific guidance.	Returns path to `verify-before-complete.txt` Custom prompt template that includes verification guidance, library-preference rules, and coding best practices alongside the standard format specification.
中间件补丁			
	PreToolUse 钩子		

（下页续）

（表 17 接上页）

C/S	组件	基础支架	最终支架（$\hat{\theta}_{\text{AS}}$）
S	`_get_completion_confirmation_message` (completion confirmation)	"Current terminal state:\n{output}\n \n Are you sure you want to mark the task as complete? This will trigger your solution to be graded and you won't be able to make any further corrections. " (generic confirmation; agent often skips verification)	"Current terminal state:\n{output}\n \n STOP – You must verify your solution before confirming completion.\n For service tasks: test endpoints, verify permissions.\n For coding tasks: test with a DIFFERENT input, verify output structure.\n Clean up: remove build artifacts the task did NOT request." (structured verification checklist with task-type-specific guidance)
	智能体循环逻辑变更		
C	`run()` (environment discovery)	Agent starts with no knowledge of what packages are pre-installed in the Docker container. Frequently chooses fragile custom implementations (e.g., regex for HTML) over established libraries that are already available.	Before the agent loop begins, `pip list --format=columns` is executed in the container so the initial terminal state shows all pre-installed Python packages. The agent can then choose appropriate libraries (e.g., BeautifulSoup over regex) from the start.

P AutoSaddler 使用的指令

为确保优化可复现且符合原则，AutoSaddler 采用多会话编排设计，每个会话都由专用提示词模板引导。这些指令旨在将我们框架的核心支柱落到实际操作中：深度调试、结构化支架干预和泛化感知选择。

诊断–补丁会话（图 13）通过让智能体立足于当前小批量中的失败来启动优化循环。与基于简单反思的方法不同，该提示词强制执行严格的"先诊断、后打补丁"工作流。它让智能体既能访问原始执行轨迹，也能访问支架代码库，并要求使用专用技能（例如 `history-analysis`、`diagnose`）来识别底层根因。此外，它将智能体限制为只能采用*能力*补丁或*引导*补丁，以此实现分阶段调度，确保干预目标明确且在架构上合理。

反思会话（图 14）聚焦因果归因和持久经验的提取。提示词要求智能体对小批量中每个场景的补丁前后执行轨迹进行对比分析。该会话的一项关键特征是强制执行"随机性检查"，智能体必须区分真正具有因果关系的修复/回归，与 LLM 非确定性造成的随机假象。该会话将结果分为四种状态（*已修复*、*发生回归*、*仍然失败*、*仍然通过*），并在 EvoDAG 中记录详细证据，从而确保只有经过验证的洞见才能驱动演化过程。

最后，**演化会话**（图 15）充当框架的元优化器。它向智能体提供 EvoDAG 历史的全局视图，包括累积经验、跨开发集的性能指标，以及先前迭代的代码差异。提示词引导智能体合成下一个候选支架：不仅可以进行增量编辑，还可能合并不同谱系中的成功组件，或撤销造成回归的更新。该会话确保所得支架基础稳定、语法和逻辑均已验证，并针对任务分布上的广泛泛化进行优化。

诊 断–补 丁 会 话 提 示 词

```
## Mandatory Skills
You MUST read and follow the SKILL.md for each skill listed below. Do NOT skip or
summarize any skill --- execute the full procedure described in each one. These skills
are installed at `.claude/skills/<n>/SKILL.md` in the current worktree.
| Order | Skill | When | Why |
|-------|-------|------|-----|
| 1 | `history-analysis` | Before any other work (Step 1) | Structured analysis of the
full evolution history --- identifies relevant lessons, prior attempts, proven
strategies, and regression-prone areas |
| 2 | `diagnose` | Before applying any patch (Step 2) | Root-cause analysis from agent
traces and codebase --- pinpoints exactly why each failing scenario fails |
| 3 | `capability-patch` or `steering-patch` | When applying patches (Step 3) |
Phase-appropriate patch methodology --- `capability-patch` during capability phase,
`steering-patch` during steering phase. Follow the skill matching the current phase |
```

```
| 4 | `patch-verification` | After applying all patches (Step 4) | Crash-safety verification --- syntax, imports, docstrings, hooks, logic. A crashing patch is worse than no patch |

> Enforcement: Every step above is mandatory and sequential. Do NOT apply patches without first completing diagnosis. Do NOT finish the session without running the full `patch-verification` procedure. Do NOT skip `history-analysis` --- it prevents repeating known failures.

## Goal

Fix failing scenarios in this mini-batch by diagnosing root causes from execution traces and the agent codebase, then applying targeted code patches --- without regressing scenarios that already pass.

Each scenario is a task the agent must solve. The initial evaluation on the mini-batch has already run the agent on each scenario and recorded its execution trace: every tool call, every response, and every reasoning step. Your job is to read these traces alongside the agent codebase, identify why the agent failed, determine what code change would fix the root cause, and apply it.

The optimization objective is to maximize performance across the full task distribution --- not just this mini-batch. Patches should address general behavioral patterns, not hardcode scenario-specific answers. A patch that fixes one scenario by adding a general rule will also help similar unseen scenarios; a patch that hardcodes a specific answer helps only that one scenario and may harm others.

Acceptance: A patch is accepted if the sum of re-evaluation scores exceeds the sum of initial scores. Even fixing a single scenario is enough. However, regressions (passing → failing) are recorded as bad patterns --- avoid them.

## Context

- Iteration: {iteration}
- Candidate: C{candidate_idx}
- Current worktree: `{worktree_path}`
- Base parent: C{base_parent_idx} (worktree: `{parent_worktree}`)
{cherry_pick_parents_section}- Phase: {phase}
- Initial evaluation output: `{before_output_dir}`

## Current Mini-Batch ({num_scenarios} scenarios)

{mini_batch_listing}

### Initial Scores (pass rate: {before_pass_rate})

{before_scores_listing}

## Patch Types for This Phase

{patch_types_section}

## Workflow

### 1. Analyze history

MANDATORY first step. Run the `history-analysis` skill to build a structured understanding of the full evolution history. Do NOT skip this step or truncate `evo-dag show history` with `head`/`tail`. The skill provides the methodology for reading the complete history without truncation. Focus on:

- Relevant lessons (good/bad patterns) for the current mini-batch
- Prior attempts on the failing scenarios --- what worked and what failed
- Proven patch strategies and regression-prone areas

### 2. Diagnose failing scenarios

For each failing scenario, use the `diagnose` skill to identify the root cause. The key is to find why the agent made the wrong decision, not just what went wrong. Diagnosis requires reading both the execution trace and the agent codebase.

Trace files (output dir: `{before_output_dir}`): See CLAUDE.md's Iteration Output Structure section for the exact file layout. Key files to read:

- Evaluation rationale: Per-scenario scores and judge rationale --- the evaluator's summary of what was expected vs what the agent produced.
- Agent execution traces: Per-scenario full tool call, response, and reasoning traces --- where you find the exact point where the agent diverged from correct behavior.

Agent codebase: current worktree `{worktree_path}`

The `diagnose` skill provides the full methodology --- follow it for each failing scenario.

### 3. Apply patches
```

Based on your diagnosis, use the phase-appropriate patch skill:

- **Capability phase** (`capability-patch`): Expand what the agent CAN DO --- new tool methods, parameter additions, implementation fixes, infrastructure changes. These unlock scenarios that are unreachable through prompt tuning alone.
- **Steering phase** (`steering-patch`): Refine HOW the agent behaves --- prompt rules, tool description corrections, PreToolUse hooks. These fine-tune the agent's use of existing capabilities.

Patch guidelines:

- **Target the root cause, not the symptom.** If the agent sends a wrong email subject, the fix is a general rule about deriving subjects from user wording --- not hardcoding the correct subject for one scenario.
- **Keep patches generalizable.** Rules should be abstract: no scenario IDs, no specific names, no hardcoded answers. A good patch helps all scenarios that share the same root cause pattern.
- **Align agent-facing text with capability changes.** When you add or modify tools, parameters, or infrastructure code, also update the system prompt, tool docstrings, and/or hooks so the agent is aware of the changes. Check for conflicting rules.
- **Learn from patch history.** Check `evo-dag show history` --- it contains diffs, per-scenario results, reflections, and accumulated lessons. Use it to understand: which patches generalized well to the dev set, which caused regressions, which root-cause patterns were effectively resolved, and which approaches repeatedly failed. Build on proven strategies and avoid repeating known bad patterns.
- **Prefer replacement over addition** when modifying prompts. The system prompt has a finite attention budget --- adding rules without removing or merging existing ones dilutes their impact.

4. Verify patches

After applying all patches, you MUST run the `patch-verification` skill to ensure the patched codebase has no runtime errors. A patch that crashes at runtime is worse than no patch --- verification is mandatory before committing. Do NOT skip this step under any circumstances.

5. Write reasoning and record intent

After applying all patches, write your reasoning to `proposer_reasoning.md` in the working directory root. For each targeted scenario, include: task description, expected vs actual behavior, trace analysis (which step went wrong), root cause, and resolution strategy (generalizable rule). This sharpens the `--diagnosis` that propagates to future iterations via `evo-dag show history`. Then record intent:

```bash
evo-dag update-intent \
  --target-scenarios "id1,id2" \
  --diagnosis "Root cause analysis of why target scenarios fail" \
  --approach "Brief description of the patch approach" \
  --files-changed "f1.py,f2.py" \
  --change-summary "What was changed and why"
```

CRITICAL: You MUST run `evo-dag update-intent` before finishing this session. Failure to do so will result in incomplete iteration records and degrade the quality of future iterations' patch history analysis.

What Happens Next

After this session, the outer loop re-evaluates the patched worktree on the same mini-batch, computes initial/re-evaluation per-scenario impacts (fixed, regressed, still_failing, still_passing), and records the patch verdict. If the patch is accepted (re-evaluation score > initial score), the candidate is also evaluated on the full dev-set to measure generalizability. Then **Session 2 (Reflection)** runs, where you analyze the results and record structured reflections for each scenario.

图 13: 诊断–补丁会话提示词：智能体接收当前小批量中的失败场景及执行轨迹，通过智能体代码库诊断根因，并对现有支架应用有针对性且可泛化的补丁。

反 思 会 话 提 示 词

```
## Mandatory Skills
You MUST read and follow the SKILL.md for each skill listed below. Do NOT skip or
summarize any skill --- execute the full procedure described in each one. These skills
are installed at `.claude/skills/<n>/SKILL.md` in the current worktree.
| Order | Skill | When | Why |
|-------|-------|------|-----|
| 1 | `history-analysis` | Before any other work (Step 1) | Structured analysis of the
full evolution history --- provides historical context for each scenario, pattern
evolution, and dev score attribution needed for meaningful reflections |
| 2 | `diagnose` | For every regressed AND fixed scenario (Step 3) | Causal attribution
--- determines whether each state change (PASS↔FAIL) was truly caused by the patch or is
a stochastic artifact from LLM non-determinism. Accurate classification is critical to
avoid polluting lessons with false signal |
> Enforcement: Do NOT attribute any state change to "stochastic noise" or "LLM
non-determinism" without running the full `diagnose` skill procedure. Every
fixed/regressed classification must cite specific evidence from the diagnosis.
## Goal
Analyze the initial/re-evaluation results for every scenario in the mini-batch and record
structured reflections that future iterations can learn from.
Your reflections feed directly into `evo-dag show history` --- the primary knowledge
store that future iterations consult when diagnosing failures, choosing patch strategies,
and avoiding past mistakes. The quality of your reflections determines how effectively
the pipeline learns. Be specific: reference exact tool calls, code paths, and behavioral
patterns. Vague reflections waste learning potential.
## Context
- Iteration: {iteration}
- Candidate: C{candidate_idx}
- Current worktree: `{worktree_path}`
- Base parent: C{base_parent_idx} (worktree: `{parent_worktree}`)
- Phase: {phase}
- Initial evaluation output: `{before_output_dir}`
- Re-evaluation output: `{train_after_cycle_dir}`
## Session 1 Reasoning
The following is the proposer's diagnosis and patch rationale from Session 1. Use this to
understand the intent behind the patch --- what root causes were identified, what the
patch was designed to fix, and what behavioral changes were expected.
{proposer_reasoning}
## Results Summary
{results_summary}
## Per-Scenario Details
{per_scenario_details}
{generalization_section}
## The Four Outcomes
| Before | After | Status | Key Question |
|--------|-------|--------|-------------|
| FAIL | PASS | `fixed` | What root cause did the patch resolve? How? |
| PASS | FAIL | `regressed` | What did the patch break? Why? |
| FAIL | FAIL | `still_failing` | Why was the patch insufficient? What to try next? |
| PASS | PASS | `still_passing` | Did the patch interact with this scenario at all? |
## Workflow
### 1. Analyze history
MANDATORY first step. Run the `history-analysis` skill to build a structured
understanding of the full evolution history. Do NOT skip this step or truncate `evo-dag
show history` with `head`/`tail`. The skill provides the methodology for reading the
complete history without truncation. Focus on:
- Historical context for each scenario in the mini-batch
```

- Pattern evolution --- emerging patterns not yet captured in lessons
- Dev score attribution --- which patches improved/hurt dev accuracy and why

This context is essential for writing meaningful reflections --- you need to know what was tried before to explain why this iteration's results differ, and to write `--prevention-or-next` guidance that adds new information rather than repeating existing lessons.

2. Read traces (by priority)

Before traces are in `{before_output_dir}`. After traces are in `{train_after_cycle_dir}`. See CLAUDE.md's **Iteration Output Structure** section for the exact file layout within each output directory.

Compare before and after traces to understand what the patch changed in the agent's behavior.

Priority order:

1. **Regressed** --- always read both before and after traces. You must understand what behavior was correct before and what the patch broke.
2. **Still-failing (targeted)** --- read to check for partial progress. Did the failure point shift? Is the agent closer to correct behavior?
3. **Fixed** --- skim the after trace to confirm the patch resolved the root cause as intended, not through a lucky side effect.
4. **Still-passing** --- skip unless the scenario uses the same tools or components you modified.

3. Diagnose state-changed scenarios (MANDATORY)

For every regressed AND fixed scenario, run the full `diagnose` skill to determine whether the state change was **truly caused by the patch** or is a **stochastic artifact** (caused by LLM non-determinism).

LLM non-determinism means the same agent code can produce different tool-call sequences across runs. This affects both directions:

- A **regressed** scenario (PASS→FAIL) may have nothing to do with the patch --- the agent simply took a different reasoning path.
- A **fixed** scenario (FAIL→PASS) may not be a real fix --- the agent may have succeeded by luck, not because of the patch. Recording this as a true fix pollutes `good_patterns` with false signal.

Accurate causal attribution is critical: false regressions accumulate noise in `bad_patterns`, and false fixes accumulate noise in `good_patterns`. Both degrade the quality of lessons for future iterations.

How to distinguish true vs. stochastic state changes:

1. **Read the before trace**: Identify the exact tool-call sequence and reasoning steps that led to the before result.
2. **Read the after trace**: Identify where the agent's behavior diverged.
3. **Check the code diff** (`evo-dag show edge {base_parent_idx} {candidate_idx}`): Does the diff touch any code path, prompt text, hook, or tool that the scenario exercises? If the diff is completely unrelated to the scenario's tool usage and divergence point, it is likely stochastic.
4. **Check the divergence point**: Is the behavioral change at a step that the patch modified (true causation) or at an unrelated step where the agent simply made a different LLM-driven choice (stochastic)?

Classification criteria:

- **True (regression or fix)**: The diff modifies code/prompt/hook that the scenario directly exercises, AND the after trace shows the agent behaving differently at the modified point in a way that explains the outcome change.
- **Stochastic**: The diff does NOT touch anything the scenario exercises, OR the agent's divergence point is unrelated to the patch (e.g., different search query phrasing, different email wording).
- **Uncertain**: The diff touches a shared component but the causal link is unclear. Record as uncertain with specific evidence.

Do NOT attribute any state change to "LLM non-determinism" or "stochastic noise" without performing the above analysis. Every classification must cite:

- The specific divergence point in the traces
- Whether the diff touches the relevant code path
- The evidence for or against a causal link

Record the classification in `--explanation` and `--prevention-or-next`. For true regressions, explain what the patch broke. For stochastic regressions, note the evidence so future iterations don't over-correct. For true fixes, explain the causal chain from patch to fix. For stochastic fixes, note the evidence so future iterations don't over-rely on the patch strategy.

4. Inspect code changes

Run `evo-dag show edge {base_parent_idx} {candidate_idx}` to see the code diff, files changed, and per-scenario impacts. Cross-reference the diff with the trace behavior to understand causality.

For deeper inspection, read source files directly in the current worktree `{worktree_path}`.

5. Record reflections

Use `evo-dag update-reflection` for **every** scenario in the mini-batch.

Fields:

- `--root-cause`: The underlying reason the scenario fails (independent of the patch). What is the agent doing wrong and why?
- `--explanation`: For `fixed` --- how the patch resolved it. For `still_failing` --- why the patch did NOT work (what was insufficient). For `regressed` --- what the patch broke and how. For `still_passing` --- why the patch did not interfere (only for scenarios using modified components).
- `--prevention-or-next`: What to try next, or what to avoid. This is critical for all non-passing outcomes --- it creates the lessons that future iterations rely on.
- `--generalization-note`: How this scenario's outcome relates to development set accuracy. Did the patch generalize beyond the mini-batch? Record when dev scores are available.

Per outcome:

`fixed` --- You MUST have completed the diagnosis step (step 3) before recording this reflection. Classify as true fix or stochastic artifact.

```bash
# True fix (patch caused the success):
evo-dag update-reflection \
  --node {candidate_idx} \
  --scenario "<id>" --status "fixed" \
  --root-cause "Agent called X with wrong param Y because docstring said Z." \
  --explanation "TRUE FIX: Patch corrected docstring to clarify Y. Agent now calls correctly at step 4 --- directly caused by diff in tools/email.py L42." \
  --prevention-or-next "For similar tool-misuse scenarios, check docstring accuracy first."
# Stochastic artifact (not caused by the patch):
evo-dag update-reflection \
  --node {candidate_idx} \
  --scenario "<id>" --status "fixed" \
  --root-cause "Agent happened to choose correct search query in after trace." \
  --explanation "STOCHASTIC: Diff only touches calendar tool. This scenario uses search tool exclusively. Agent succeeded because it picked a better search query by chance at step 2 --- unrelated to patch." \
  --prevention-or-next "Not a reliable fix --- scenario may fail again. Root cause (weak search strategy) still needs addressing."
```

`regressed` (highest priority) --- You MUST have completed the diagnosis step (step 3) before recording this reflection. Classify as true regression or stochastic artifact with evidence.

```bash
# True regression (patch caused the failure):
evo-dag update-reflection \
  --node {candidate_idx} \
  --scenario "<id>" --status "regressed" \
  --root-cause "This scenario relied on optional param Y in tool X." \
```

```
      --explanation "TRUE REGRESSION: Docstring change removed note about Y being
optional. Agent stopped passing Y. Divergence at step 5 where agent no longer passes Y
--- directly caused by diff in tools/email.py L42." \
      --prevention-or-next "When modifying docstrings, preserve param optionality
annotations."
    # Stochastic artifact (not caused by the patch):
    evo-dag update-reflection \
      --node {candidate_idx} \
      --scenario "<id>" --status "regressed" \
      --root-cause "Agent used different search query phrasing in after trace." \
      --explanation "STOCHASTIC: Diff only touches calendar tool docstring. This scenario
uses email tool exclusively. Agent diverged at step 3 with different query wording ---
unrelated to patch." \
      --prevention-or-next "No action needed --- stochastic noise. Do not over-correct
for this scenario."
    ```
`still_failing` --- Explain why the patch was insufficient. What should the next
iteration try instead?
    ```bash
    evo-dag update-reflection \
      --node {candidate_idx} \
      --scenario "<id>" --status "still_failing" \
      --root-cause "Agent uses absolute dates instead of relative dates in messages." \
      --explanation "Prompt rule 'use relative dates' was too weak --- agent still used
'Oct 22'. Partial progress: bullet formatting was fixed." \
      --prevention-or-next "Stronger rule needed. Consider PreToolUse hook on
send_message for just-in-time reminder."
    ```
`still_passing` --- For scenarios that use the same tools or components you modified,
explain which specific patch change could have caused interference and why it didn't.
This is valuable signal for understanding patch safety.
    ```bash
    evo-dag update-reflection \
      --node {candidate_idx} \
      --scenario "<id>" --status "still_passing" \
      --explanation "Uses send_message but passes because it sends user-provided verbatim
content. Our rule only affects agent-composed messages."
    ```
For scenarios unrelated to the patch, use the batch command:
    ```bash
    evo-dag update-reflection --node {candidate_idx} --batch-still-passing "id1,id2,..."
    ```
{generalization_workflow_step}
Causal Attribution Checklist
Before finishing, verify each regressed AND fixed scenario's reflection:
1. Classified: Explicitly labeled as TRUE (REGRESSION/FIX) or STOCHASTIC
2. Evidence-based: Cites the specific divergence point in traces
3. Diff-linked: States whether the diff touches the relevant code path
4. Actionable: True regressions have prevention guidance; true fixes explain the causal
chain; stochastic cases explicitly state the evidence and implications
Reflection Quality Checklist
Before finishing, verify each reflection meets these criteria:
1. Specific: References exact tool calls, steps, or code paths
2. Causal: Explains WHY the outcome occurred, not just WHAT happened
3. Actionable: `--prevention-or-next` gives clear guidance for future iterations --- not
generic advice like "try harder"
4. Distinct: Each reflection adds unique information (no copy-paste templates across
scenarios)
```
```

```
## Constraints
- Record a reflection for every scenario in the mini-batch.
- Use `evo-dag update-reflection` --- do NOT write reflections to files.
- Do NOT modify source code during reflection --- Session 2 is analysis only.
- Do NOT read raw traces from development set output directories --- only aggregate dev accuracy is available for generalization analysis.
- Do NOT read raw traces from development set dirs (`_val_` or `seed_val_` output dirs) --- only aggregate accuracy is available.
CRITICAL: You MUST run `evo-dag update-reflection` for every scenario in the mini-batch before finishing this session. Use individual calls for fixed/regressed/still_failing scenarios and `--batch-still-passing` for unaffected passing scenarios. Missing reflections degrade the quality of accumulated lessons for future iterations.
```

图 14: 反思会话提示词：智能体逐场景分析初始评估与重新评估的结果（fixed / regressed / still_failing / still_passing），区分补丁的真实因果效应与 LLM 非确定性造成的随机现象，并记录结构化反思，将其反馈到 evo-dag show history 中供后续迭代使用。

演 化 会 话 提 示 词

```
## Mandatory Skills
You MUST read and follow the SKILL.md for each skill listed below. Do NOT skip or summarize any skill --- execute the full procedure described in each one. These skills are installed at `.claude/skills/<n>/SKILL.md` in the current worktree.
| Order | Skill | When | Why |
|-------|-------|------|-----|
| 1 | `history-analysis` | Before any other work (Step 1) | Structured analysis of the full evolution history --- classifies every patch as revert vs. preserve to inform candidate selection |
| 2 | `patch-verification` | After any codebase change (Step 5) | Verifies the prepared base codebase has no runtime errors --- a broken base causes every subsequent session to fail |
> Enforcement: If you make any change to the worktree (rsync, cherry-pick, revert, or code edit), you MUST run the `patch-verification` skill's full procedure before finishing. Skipping verification is a critical failure.
## Goal
Prepare the best possible base codebase for this iteration's patch. The optimization objective is to maximize performance across the task distribution, with development set (dev) accuracy as the proxy.
Your selection must be data-driven, based on dev scores, regression analysis, and cross-candidate code comparison --- not on attachment to accumulated patches.
Anti--Sunk-Cost Principle: The number of iterations invested in a lineage is NOT a reason to continue it. A long lineage with declining dev scores is a signal to switch, not to persist. Evaluate each candidate by its measured performance and the quality of its code, not by how many patches led to it.
## Context
- Iteration: {iteration}
- Phase: {phase}
- Current worktree: `{worktree_path}` (fork of C{base_parent_idx})
- Base parent: C{base_parent_idx} (worktree: `{parent_worktree}`)
## Candidate Performance (sorted by dev score)
{candidate_table}
Note: Dev scores are only available for candidates that passed mini-batch acceptance. "pending" means the candidate was not evaluated on the dev set. Use patch history, raw traces, and codebase inspection to assess these.
## DAG Topology
{dag_topology}
## Selection Strategy
Choose the base candidate by weighing dev scores, regression history, score trajectory, and code quality together. Dev scores are noisy --- small differences may be evaluation
```

variance, so corroborate with regression counts and code inspection. Do not stay on a lineage just because many patches were accumulated; patches only have value if performance improved.

Options:

- **Continue from latest**: When the lineage is near its peak and stable. Revert any regressions and cherry-pick fixes from other candidates.
- **Switch parent**: When dev score has dropped well below the best candidate and the trajectory is not recovering. Cherry-pick verified good patches from the old lineage.
- **Combine candidates**: When different candidates have complementary strengths in non-overlapping areas.

Key rules:

- Regressions identified here must be reverted in THIS session. Do NOT defer to Session 1 --- Session 1 only addresses the current mini-batch and will not fix prior regressions.
- When cherry-picking, prefer diff-level precision over copying entire files. Use `evo-dag show edge` to see exact diffs and apply only the relevant changes. Whole-file copies via rsync bring unintended changes.
- Compare top candidates' codebases directly (`diff -rq`) to find cherry-pick opportunities that aren't visible from scores alone.

Workflow

1. Analyze history

MANDATORY first step. Run the `history-analysis` skill to build a structured understanding of the full evolution history. Do NOT skip this step or truncate `evo-dag show history` with `head`/`tail`. The skill provides the methodology for reading the complete history without truncation.

The purpose of this analysis is to **classify every patch** in the history into two categories that directly inform candidate selection:

Patches to revert (caused severe regressions)

- Which patches introduced regressions? Identify the exact iteration, candidate, and code diff that caused each regression.
- How severe is each regression? (How many scenarios regressed? Did dev score drop?)
- Is the regression still present in the current lineage, or was it already fixed by a later patch?
- What files were modified --- so you know exactly what to revert or undo.

Patches to preserve (drove performance gains)

- Which patches produced the largest improvements in dev score?
- Which patches fixed scenarios that stayed fixed in subsequent iterations (durable fixes vs. fragile ones)?
- Which patches generalized well beyond their target mini-batch?
- What files and patterns were involved --- so you know what to protect when combining candidates.

This classification directly drives your selection decision:

- **Continue from latest**: Revert the harmful diffs while keeping the beneficial ones.
- **Switch parent**: If too many regressions have accumulated, switch to the best candidate and cherry-pick the preserved patches.
- **Combine candidates**: If different candidates have complementary preserved patches, combine them.

2. Investigate candidates

Use the following sources to decide which candidate(s) to build on:

1. **History analysis output**: The structured summary from Step 1 --- lineage trajectory, lessons, and cross-candidate strengths.
2. **Specific candidate**: `evo-dag show node <idx>` --- scores, patch intent, verdict, output directories.
3. **Raw traces**: Read evaluation output and trace files in the iteration output directories listed by `evo-dag show node` or `evo-dag show current-batch`. See CLAUDE.md's **Iteration Output Structure** section for the file layout.
4. **Candidate codebases**: Read source files directly in other candidates' worktrees to understand what changed and whether a fix is worth porting. Worktree paths are shown in the candidate table and `evo-dag show node`.

5. **Cross-candidate code comparison (MANDATORY for top candidates)**: For the top 2-3 candidates by dev score, directly compare their codebases to identify divergent files and cherry-pick opportunities:

    ```bash
    diff -rq {session_root}/worktrees/<candidate_A_dir>/
{session_root}/worktrees/<candidate_B_dir>/ | grep -v '.git'
    ```

   Then read the divergent files in both worktrees to understand which candidate's version is better for each file. This comparison reveals cherry-pick opportunities that are invisible from score data alone.

3. Revert regressions (MANDATORY if continuing from latest)

If you chose to continue from the latest candidate, you MUST revert all identified regressions in THIS session. Do NOT defer regression fixes to Session 1 --- Session 1 only addresses the current mini-batch and will not fix prior regressions.

For each regression identified in Step 1:

1. **Locate the exact diff** that caused the regression using `evo-dag show edge <parent> <child>` for the iteration that introduced it.

2. **Compare with pre-regression code**: Read the same file in the pre-regression candidate's worktree to see what the code looked like before the harmful change.

3. **Revert only the harmful lines**: Do not revert the entire patch if it also contains beneficial changes. Surgically revert only the lines that caused the regression.

4. **Preserve beneficial changes**: If a patch contains both good and bad changes, keep the good parts. Check whether specific scenarios were fixed by specific lines in the diff to determine what to preserve.

After reverting, run the `patch-verification` skill to ensure no runtime errors were introduced.

4. Prepare the base codebase

Each candidate has a **base parent** (the candidate it was forked from) and optionally **cherry-pick parents** (other candidates whose code was referenced, copied, or combined). The base parent is set automatically by the outer loop. Cherry-pick parents are what you record here.

Your worktree starts as a fork of C{base_parent_idx} (base parent).

Switching base entirely

If you decided to switch base, copy the new base's entire codebase:

    ```bash
    rsync -a --exclude='.git' {session_root}/worktrees/<candidate_dir>/ ./
    ```

Then cherry-pick good patches from the old lineage using the diff-level method below.

Diff-level cherry-pick (preferred over file-level copy)

When porting specific fixes from another candidate, use **diff-level precision** rather than copying entire files. Copying entire files brings unintended changes from the source candidate.

1. **Identify the exact diff**: Use `evo-dag show edge <parent> <child>` to see the exact code changes that constituted the fix you want to port.

2. **Compare files**: Read the specific file in both the source candidate's worktree and your current worktree. Use `diff` to see differences:

    ```bash
    diff {session_root}/worktrees/<source_candidate_dir>/<file> ./<file>
    ```

3. **Apply only the relevant changes**: Manually apply only the lines from the diff that correspond to the fix. Do not copy unrelated changes that the source candidate may have accumulated.

4. **Verify**: After each cherry-pick, run the `patch-verification` skill.

5. Verify changes

If you made any changes to the worktree (rsync, cherry-pick, revert, etc.), you MUST run the `patch-verification` skill to ensure the codebase has no runtime errors. A broken base codebase will cause every subsequent session to fail --- verification is mandatory.

6. Record selection

```
After preparing the base code, record your selection with `evo-dag update-selection`.
List all candidates you referenced --- whether you switched base entirely, cherry-picked
specific files, or just used their code as reference for a re-implementation:
    ```bash
    evo-dag update-selection \
      --parent-candidates "<idx1>,<idx2>,..." \
      --reasoning "<what you took from each and why>"
    ```
CRITICAL: You MUST run `evo-dag update-selection` before finishing this session. Failure
to do so will result in lost selection history that future iterations need to understand
the DAG lineage.
What Happens Next
After this session, the outer loop runs Session 1 (Diagnose + Patch): the agent receives
the current mini-batch's failing scenarios with execution traces, diagnoses root causes,
and applies targeted code patches to the worktree you prepared here.
```

图 15: 演化会话提示词：由数据驱动地从先前候选方案中选择基础代码库，包括历史分析、回归撤销和差异级拣选。

Q 数据许可证

本文所用数据集的许可证如下：

- GAIA2：cc-by-4.0
- SWE-Bench Pro：MIT 许可证
- Terminal-Bench 2.0：Apache-2.0 许可证

所有数据集均严格用于研究目的，未在任何非研究情境中使用，尤其未用于商业应用。

R 局限性与未来工作

AutoSaddler 目前将支架优化表述为一个监督学习问题，假设能够获得由任务及其标准答案组成的训练集，以及为每次智能体执行提供明确信号的任务级成功指标（例如通过/失败）。尽管这一假设符合标准智能体基准，但现实部署环境中可能缺乏此类真实结果，而大规模获取这些结果的成本可能高得难以承受。因此，一个自然的扩展方向是放宽这一监督要求，研究无监督或弱监督支架优化；在这类设定中，失败信号来自轨迹级不一致、工具调用错误或跨多次执行的自洽性等内在提示，而不是外部标签。另一个方向是从智能体环境中自举合成训练数据，例如 GAIA2 中每位用户的世界。最后，我们目前的范围仅限于无状态、相互独立的任务；将 AutoSaddler 扩展到有状态设定，整合记忆与技能策展，并在更广泛的 LLM 家族上验证该方法，仍是未来工作的重要方向。
就现实部署而言，我们要强调在开发早期采用强治理和人在回路机制的重要性：自动支架修改在部署到生产系统之前，应接受人工审查或额外的安全验证。